Slip1

1)

```
/*Program for LFU Page replacement simulation*/
#include<stdio.h>

struct node
{
  int pno,freq;
}frames[20];
int n;

int page_found(int pno)
{
  int fno;
  for(fno=0;fno<n;fno++)
            if(frames[fno].pno==pno)
               return fno;
            return -1;
}

int get_free_frame()
{  int fno;
  for (fno=0; fno<=n; fno++)
            if (frames[fno].pno==-1)
                        return(fno);
   return(-1);
}

int get_lfu_frame()
{
  int fno;
```

```
  int selfno=0;
  for (fno=1; fno<n; fno++)
            if(frames[fno].freq<frames[selfno].freq)
            selfno=fno;
  return selfno;
}
int main()
{
   int p_request[]={3,4,5,4,3,4,7,2,4,5,6,7,2,4,6};
   int size=12;
   int page_falts=0,i,j,fno;

   printf("\nHow many frames:");  scanf("%d",&n);
   //initialize frames
   for (i=0; i<n; i++)
   { frames[i].pno=-1;
             frames[i].freq=0;
   }

   printf("\nPageNo     Page Frames          Page Fault");
   printf("\n-------------------------------------------------");
   for(i=0;i<size;i++)
   {
             j=page_found(p_request[i]);
             if(j==-1)  //page fault occurs
             {
               j=get_free_frame();
               if (j==-1) //no free frame - do page replacement
                         j=get_lfu_frame();
               page_falts++;
               frames[j].pno=p_request[i];
```

```
            frames[j].freq=1;
            printf("\n%4d\t ",p_request[i]);
            for (fno=0; fno<n; fno++)
                    printf("%4d:%2d",frames[fno].pno,frames[fno].freq);
            printf(" : YES");
           }
          else //page found in frame j
          {
            printf("\n%4d\t ",p_request[i]);
            frames[j].freq++;
            for (fno=0; fno<n; fno++)
                    printf("%4d:%2d",frames[fno].pno,frames[fno].freq);
            printf(" : NO");
          }
  }
 printf("\n-----------------------------------------------------");
 printf("\n Number of Page_Falts=%d",page_falts);
// getch();
}
```

2)

```
#include <stdio.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <dirent.h>
//#include <bfd.h>
char comm[100],*ptr;
char *args[10];
```

```
int tot_arg;

int typeline( char *option, char *fname);

main()
{
	do
	{
		printf("myshell $ ");
		getcomm();
		sep_arg();
		take_action();
	}while(1);
} // main

sep_arg()
{
	int i,j;
	i = j = tot_arg = 0;

	args[tot_arg] = (char*)malloc(sizeof(char) * 20);
	for(i=0; comm[i] !='\0' ; i++)
	{
		if( comm[i] == ' ')
		{
			args[tot_arg][j] = '\0';
			tot_arg++;
			args[tot_arg] = (char*)malloc(sizeof(char) * 20);
			j=0;
		}
		else
```

```
                {
                        args[tot_arg][j] =comm[i];
                        j++;
                }
        } // while
        args[tot_arg][j] = '\0'; // complete last word
        return;
}

getcomm()
{
        int len;
        ptr = fgets(comm,80,stdin);
        len = strlen(comm);
        comm[len-1] = '\0';
        return;
} // getcomm

take_action()
{
        char str[100];
        pid_t   pid;
        int status;

        if(strcmp(args[0],"typeline") == 0 )
                typeline(args[1],args[2]);
        else
        {
                if( ptr == NULL )
                        exit(0);
                if ( strlen(comm) <= 1 )
```

```
                return;
        if( ( pid = fork() ) < 0 )
        {
                printf("\nUnable to create process");
        }
        else
        if(  pid == 0 ) // child
        {
    sscanf(comm,"%s",comm);
                execvp(comm,args);
                strcpy(str,"./");
                strcat(str,args[0]);
                execvp(str,args);
                printf("\n%s : command not found",comm);
                exit(0);
        }
        else //parent
         waitpid(pid,&status,0);
    } // else
}// take action

int typeline( char *option, char *fname)
{
    FILE *fp;
    int tot_lines,ch,n,cnt;
    //---- count the lines
    fp= fopen(fname,"r");
    if( fp == NULL )
    {
        printf("\nUnable to open file");
        exit(0);
```

```
}
tot_lines = 0;//count total lines
ch = fgetc(fp);
while ( ch != EOF )
{
        if( ch == 10 ) //new line char
        tot_lines++;
        ch = fgetc(fp);
}
fclose(fp);
//---- take action
if( strcmp(option,"a")== 0 ) // print all lines
{
        fp= fopen(fname,"r");
        ch = fgetc(fp);
        while ( ch != EOF )
        {
                printf("%c",ch);
                ch = fgetc(fp);
        }
        fclose(fp);
}
else
if( option[0] =='+') //print first n lines)
{
        n = atoi(option+1);
        if( n > tot_lines)
        {
                printf("\nInvalid  option");
                exit(0);
        }
```

```
        fp= fopen(fname,"r");
        cnt = 0;
        while( cnt < n )
        {
                ch = fgetc(fp);
                printf("%c",ch);
                if( ch == 10)
                        cnt++;
        }
        fclose(fp);
}
else
if( option[0] =='-') //print last  n lines)
{
        n = atoi(option+1);
        if( n > tot_lines)
        {
                printf("\nInvalid  option");
                exit(0);
        }
        fp= fopen(fname,"r");
        cnt = 0;
        while( cnt < tot_lines -n )
        {
                ch = fgetc(fp);
                if( ch == 10)
                        cnt++;
        }
        while( (ch=fgetc(fp)) != EOF )
                printf("%c",ch);
        fclose(fp);
```

```
        }
        else
        {
                printf("\nInvalid option");
                exit(0);
        }
        return(0);
} //typeline
```

Slip2

1)

```
/*Program for FIFO*/
#include<stdio.h>

int frame[3]={-1,-1,-1};
int pg_found(int pno)
{
        int i;
        for(i=0;i<3;i++)
                if(frame[i]==pno)
                return i;
        return -1;
}
int main()
{
        int p_req[]={3,4,5,6,3,4,7,3,4,5,6,7,2,4,6};
        int size=15,page_fault=0,i;

        printf("\n FIFO page replacement");
        printf("\n page no  -----------------page frames");
        printf("\n--------------------------------------");
```

```
	for(i=0;i<size;i++)
	{
		if(pg_found(p_req[i])==-1)
		{
			frame[page_fault%3]=p_req[i];
			page_fault++;
			printf("\n %4d--------------------
%4d%4d%4d",p_req[i],frame[0],frame[1],frame[2]);
		}
		else
			printf("\n %4d.......No Page fault",p_req[i]);
	}
	printf("\n-----------------------------------------");
	printf("\n Total page fault=%d",page_fault);

}
```

2)

```
#include <stdio.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <dirent.h>
//#include <bfd.h>
char comm[100],*ptr;
char *args[10];
int tot_arg;

int list( char option, char dirname[]);
```

```
main()
{
	do
	{
		printf("myshell $ ");
		getcomm();
		sep_arg();
		take_action();
	}while(1);
} // main

sep_arg()
{
	int i,j;
	i = j = tot_arg = 0;

	args[tot_arg] = (char*)malloc(sizeof(char) * 20);
	for(i=0; comm[i] !='\0' ; i++)
	{
		if( comm[i] == ' ')
		{
			args[tot_arg][j] = '\0';
			tot_arg++;
			args[tot_arg] = (char*)malloc(sizeof(char) * 20);
			j=0;
		}
		else
		{
			args[tot_arg][j] =comm[i];
			j++;
```

```
                }
        } // while
        args[tot_arg][j] = '\0'; // complete last word
        return;
}

getcomm()
{
        int len;
        ptr = fgets(comm,80,stdin);
        len = strlen(comm);
        comm[len-1] = '\0';
        return;
} // getcomm

take_action()
{
        char str[100];
        pid_t   pid;
        int status;

        if(strcmp(args[0],"list") == 0 )
                list(*args[1],args[2]);
        else
        {
                if( ptr == NULL )
                        exit(0);
                if ( strlen(comm) <= 1 )
                        return;
                if( ( pid = fork() ) < 0 )
                {
```

```
                printf("\nUnable to create process");
            }
            else
            if(  pid == 0 ) // child
            {   sscanf(comm,"%s",comm);
                execvp(comm,args);
                strcpy(str,"./");
                strcat(str,args[0]);
                execvp(str,args);
                printf("\n%s : command not found",comm);
                exit(0);
            }
            else //parent
             waitpid(pid,&status,0);
        } // else
}// take action

int list( char option, char dirname[])
{
        DIR *dp;
        struct dirent *dent;
        struct stat buff;
        char orgdir[100];
        int cnt,reply;
        getcwd(orgdir,90);
        chdir(dirname);
        switch(option)
        {
        case 'f' : // list all dir entries
                dp= opendir(".");
                if( dp == NULL )
```

```
                {
                        printf("\nUnable to open dir");
                        exit(0);
                }
                dent = readdir(dp);
                while( dent != NULL )
                {
                        printf("\n%s",dent->d_name);
                        dent= readdir(dp);
                }
                printf("\n");
                break;
case 'n' : // list number all dir entries
                cnt =0;
                dp= opendir(".");
                if( dp == NULL )
                {
                        printf("\nUnable to open dir");
                        exit(0);
                }
                dent = readdir(dp);
                while( dent != NULL )
                {
                        cnt++;
                        dent= readdir(dp);
                }
                printf("\ntot dir entries = %d\n",cnt);
                closedir(dp);
                break;
default :       printf("\nInvalid option");
                exit(0);
```

```
        } // switch
        chdir(orgdir);
        return(0);
}// main
```

Slip3

1)

```
/*Program for LRU Page replacement simulation*/
#include<stdio.h>
struct node
{
  int pno,reftime;
}frames[20];
int n;

int page_found(int pno)
{
  int fno;
  for(fno=0;fno<n;fno++)
        if(frames[fno].pno==pno)
          return fno;
        return -1;
}

int get_free_frame()
{  int fno;
  for (fno=0; fno<=n; fno++)
        if (frames[fno].pno==-1)
                return(fno);
   return(-1);
}
```

```
int get_lru_frame()
{
  int fno;
  int selfno=0;
  for (fno=1; fno<n; fno++)
            if(frames[fno].reftime<frames[selfno].reftime)
            selfno=fno;
  return selfno;
}
main()
{
   int p_request[]={3,5,7,2,5,1,2,3,1,3,5,3,1,6,2};
   int size=15,currtime;
   int page_falts=0,i,j,fno;

   printf("\nHow many frames:");  scanf("%d",&n);
   //initialize frames
   for (i=0; i<n; i++)
   { frames[i].pno=-1;
             frames[i].reftime=-1;
   }

   printf("\nPageNo     Page Frames         Page Fault");
   printf("\n-----------------------------------------");
   currtime=0;
   for(i=0;i<size;i++)
   {
             j=page_found(p_request[i]);
             if(j==-1)  //page fault occurs
             {
```

```
            j=get_free_frame();
            if (j==-1) //no free frame - do page replacement
                    j=get_lru_frame();
            page_falts++;
            frames[j].pno=p_request[i];
            frames[j].reftime=currtime;
            printf("\n%4d\t ",p_request[i]);
            for (fno=0; fno<n; fno++)
                    printf("%4d",frames[fno].pno);
            printf(" : YES");
           }
          else//page found
          {
            printf("\n%4d\t ",p_request[i]);
            frames[j].reftime=currtime;
            for (fno=0; fno<n; fno++)
                    printf("%4d",frames[fno].pno);
            printf(" : NO");
          }
            currtime++;
  }
 printf("\n----------------------------------------");
 printf("\n Number of Page_Falts=%d",page_falts);

}
```

2)

```
#include <stdio.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>
```

```
#include <sys/stat.h>
#include <fcntl.h>
#include <dirent.h>
//#include <bfd.h>
char comm[100],*ptr;
char *args[10];
int tot_arg;

int count( char option, char *fname);

main()
{
	do
	{
		printf("myshell $ ");
		getcomm();
		sep_arg();
		take_action();
	}while(1);
} // main

sep_arg()
{
	int i,j;
	i = j = tot_arg = 0;

	args[tot_arg] = (char*)malloc(sizeof(char) * 20);
	for(i=0; comm[i] !='\0' ; i++)
	{
		if( comm[i] == ' ')
		{
```

```
                        args[tot_arg][j] = '\0';
                        tot_arg++;
                        args[tot_arg] = (char*)malloc(sizeof(char) * 20);
                        j=0;
                }
                else
                {
                        args[tot_arg][j] =comm[i];
                        j++;
                }
        } // while
        args[tot_arg][j] = '\0'; // complete last word
        return;
}

getcomm()
{
        int len;
        ptr = fgets(comm,80,stdin);
        len = strlen(comm);
        comm[len-1] = '\0';
        return;
} // getcomm

take_action()
{
        char str[100];
        pid_t   pid;
        int status;

        if(strcmp(args[0],"count") == 0 )
```

```
                count(*args[1],args[2]);
        else
        {
                if( ptr == NULL )
                        exit(0);
                if ( strlen(comm) <= 1 )
                        return;
                if( ( pid = fork() ) < 0 )
                {
                        printf("\nUnable to create process");
                }
                else
                if(  pid == 0 ) // child
                {   sscanf(comm,"%s",comm);
                        execvp(comm,args);
                        strcpy(str,"./");
                        strcat(str,args[0]);
                        execvp(str,args);
                        printf("\n%s : command not found",comm);
                        exit(0);
                }
                else //parent
                 waitpid(pid,&status,0);
        } // else
}// take action

int count( char option, char *fname)// count c/w/l   <filename>
{
        int ccnt,wcnt,lcnt,wflag;
        int ch;
        FILE *fp;
```

```
int i;
fp= fopen(fname,"r");
if( fp == NULL )
{
        printf("\nUnable to open file");
        exit(0);
}
ccnt =wcnt = lcnt = 0;
wflag=1;
ch = fgetc(fp);
while ( ch != EOF)
{
        ccnt++;
        if( ch ==' ' || ch== '\t')
        {
                if( wflag == 0 )
                {
                        wcnt++;
                        wflag = 1;
                }
        }
        else
        if( ch == '\n')
        {
                lcnt++;
                if( wflag == 0 )
                {
                        wcnt++;
                        wflag = 1;
                }
        }
```

```
		else
		{
			wflag = 0;
		}
		ch = fgetc(fp);
	}//while
	if( wflag == 0 )
	{
		wcnt++;
		lcnt++;
	}
	fclose(fp);
	switch(option)
	{
	case 'c' : // print all chars
			printf("\nTot chars = %d\n",ccnt);
			break;
	case 'w' : // print all words
			printf("\nTot words = %d\n",wcnt);
			break;
	case 'l' : // print all lines
			printf("\nTot lines = %d\n",lcnt);
			break;
	} // switch
	return;
}// count
```

Slip4

1)

```
/*Program for MFU Page replacement simulation*/
#include<stdio.h>
```

```
struct node
{
  int pno,freq;
}frames[20];
int n;

int page_found(int pno)
{
  int fno;
  for(fno=0;fno<n;fno++)
            if(frames[fno].pno==pno)
              return fno;
            return -1;
}

int get_free_frame()
{  int fno;
  for (fno=0; fno<=n; fno++)
            if (frames[fno].pno==-1)
                        return(fno);
   return(-1);
}

int get_mfu_frame()
{
  int fno;
  int selfno=0;
  for (fno=1; fno<n; fno++)
            if(frames[fno].freq>frames[selfno].freq)
            selfno=fno;
```

```
  return selfno;
}
int main()
{
   int p_request[]={8,5,7,8,5,7,2,3,7,3,5,9,4,6,2};
   int size=12;
   int page_falts=0,i,j,fno;

   printf("\nHow many frames:");  scanf("%d",&n);
   //initialize frames
   for (i=0; i<n; i++)
   { frames[i].pno=-1;
            frames[i].freq=0;
   }

   printf("\nPageNo     Page Frames          Page Fault");
   printf("\n--------------------------------------------------");
   for(i=0;i<size;i++)
   {
            j=page_found(p_request[i]);
            if(j==-1)  //page fault occurs
            {
              j=get_free_frame();
              if (j==-1) //no free frame - do page replacement
                       j=get_mfu_frame();
              page_falts++;
              frames[j].pno=p_request[i];
              frames[j].freq=1;
              printf("\n%4d\t ",p_request[i]);
              for (fno=0; fno<n; fno++)
                       printf("%4d:%2d",frames[fno].pno,frames[fno].freq);
```

```
            printf(" : YES");
           }
          else //page found in frame j
          {
            printf("\n%4d\t ",p_request[i]);
            frames[j].freq++;
            for (fno=0; fno<n; fno++)
                    printf("%4d:%2d",frames[fno].pno,frames[fno].freq);
            printf(" : NO");
          }
  }
 printf("\n-----------------------------------------------------");
 printf("\n Number of Page_Falts=%d",page_falts);
// getch();
}
```

2)

```
#include <stdio.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <dirent.h>
//#include <bfd.h>
char comm[100],*ptr;
char *args[10];
int tot_arg;

int search(char option, char *pattern, char *fname);
```

```
main()
{
	do
	{
		printf("myshell $ ");
		getcomm();
		sep_arg();
		take_action();
	}while(1);
} // main

sep_arg()
{
	int i,j;
	i = j = tot_arg = 0;

	args[tot_arg] = (char*)malloc(sizeof(char) * 20);
	for(i=0; comm[i] !='\0' ; i++)
	{
		if( comm[i] == ' ')
		{
			args[tot_arg][j] = '\0';
			tot_arg++;
			args[tot_arg] = (char*)malloc(sizeof(char) * 20);
			j=0;
		}
		else
		{
			args[tot_arg][j] =comm[i];
			j++;
		}
```

```
        } // while

        args[tot_arg][j] = '\0'; // complete last word
        return;
}

getcomm()
{
        int len;
        ptr = fgets(comm,80,stdin);
        len = strlen(comm);
        comm[len-1] = '\0';
        return;
} // getcomm

take_action()
{
        char str[100];
        pid_t   pid;
        int status;

        if(strcmp(args[0],"search") == 0 )
                search(*args[1],args[2],args[3]);
        else
        {
                if( ptr == NULL )
                        exit(0);
                if ( strlen(comm) <= 1 )
                        return;
                if( ( pid = fork() ) < 0 )
                {
```

```
                printf("\nUnable to create process");
            }
            else
            if(  pid == 0 ) // child
            {   sscanf(comm,"%s",comm);
                execvp(comm,args);
                strcpy(str,"./");
                strcat(str,args[0]);
                execvp(str,args);
                printf("\n%s : command not found",comm);
                exit(0);
            }
            else //parent
             waitpid(pid,&status,0);
        } // else
}// take action

int search(char option, char *pattern, char *fname)
{
        char buff[100],*ptr;
        int cnt;
        FILE *fp;
        int i;
        fp= fopen(fname,"r");
        if( fp == NULL )
        {
            printf("\nUnable to open file");
            exit(0);
        }
        switch(option)
        {
```

```
        case 'a' : // print all occ
                        while ( !feof (fp))
                        {
                                buff[0] = '\0';
                                fgets(buff,80,fp);
                                ptr = strstr(buff,pattern);
                                if( ptr != NULL )
                                        printf("%s",buff);
                        }
                        fclose(fp);
                        break;
        case 'c' : // count occ
                        cnt=0;
                        while ( !feof (fp))
                        {
                                buff[0] = '\0';
                                fgets(buff,80,fp);
                                ptr = strstr(buff,pattern);
                                while(ptr != NULL )
                                {
                                        cnt++;
                                        ptr = ptr + strlen(pattern);
                                        ptr = strstr(ptr,pattern);
                                } // while
                        }
                        fclose(fp);
                        printf("\nThe serach string %s occurs %d times",pattern,cnt);
                        break;
    } // switch
    return;
}//search
```

Slip5

1)

2)

```
#include <stdio.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <dirent.h>
//#include <bfd.h>
char comm[100],*ptr;
char *args[10];
int tot_arg;

int search(char option, char *pattern, char *fname);

main()
{
	do
	{
		printf("myshell $ ");
		getcomm();
		sep_arg();
		take_action();
	}while(1);
} // main

sep_arg()
```

```
{
        int i,j;
        i = j = tot_arg = 0;

        args[tot_arg] = (char*)malloc(sizeof(char) * 20);
        for(i=0; comm[i] !='\0' ; i++)
        {
                if( comm[i] == ' ')
                {
                        args[tot_arg][j] = '\0';
                        tot_arg++;
                        args[tot_arg] = (char*)malloc(sizeof(char) * 20);
                        j=0;
                }
                else
                {
                        args[tot_arg][j] =comm[i];
                        j++;
                }
        } // while
        args[tot_arg][j] = '\0'; // complete last word
        return;
}



getcomm()
{
        int len;
        ptr = fgets(comm,80,stdin);
        len = strlen(comm);
```

```
        comm[len-1] = '\0';
        return;
} // getcomm

take_action()
{
        char str[100];
        pid_t   pid;
        int status;

        if(strcmp(args[0],"search") == 0 )
                search(*args[1],args[2],args[3]);
        else
        {
                if( ptr == NULL )
                        exit(0);
                if ( strlen(comm) <= 1 )
                        return;
                if( ( pid = fork() ) < 0 )
                {
                        printf("\nUnable to create process");
                }
                else
                if(  pid == 0 ) // child
                {   sscanf(comm,"%s",comm);
                        execvp(comm,args);
                        strcpy(str,"./");
                        strcat(str,args[0]);
                        execvp(str,args);
                        printf("\n%s : command not found",comm);
                        exit(0);
```

```
                }
                else //parent
                 waitpid(pid,&status,0);
        } // else
}// take action

int search(char option, char *pattern, char *fname)
{
        char buff[100],*ptr;
        int cnt;
        FILE *fp;
        int i;
        fp= fopen(fname,"r");
        if( fp == NULL )
        {
                printf("\nUnable to open file");
                exit(0);
        }
        switch(option)
        {
           case 'c' : // count occ
                        cnt=0;
                        while ( !feof (fp))
                        {
                                buff[0] = '\0';
                                fgets(buff,80,fp);
                                ptr = strstr(buff,pattern);
                                while(ptr != NULL )
                                {
                                        cnt++;
                                        ptr = ptr + strlen(pattern);
```

```
                        ptr = strstr(ptr,pattern);
                    } // while
                }
                fclose(fp);
                printf("\nThe serach string %s occurs %d times",pattern,cnt);
                break;

        case 'f' : //print first occ
                while ( !feof (fp))
                {
                    buff[0] = '\0';
                    fgets(buff,80,fp);
                    ptr = strstr(buff,pattern);
                    if( ptr != NULL )
                    {
                        printf("%s",buff);
                        break;
                    }
                }
                fclose(fp);
                break;
            } // switch
        return;
}//search
```

Slip6

1)

```
/*Program for mru*/

#include<stdio.h>
#include<conio.h>
```

```
struct frmnode
{
 int pno,counter;
}
frames[20];

int n;

int page_found(int pno)
{
 int fno;
 for(fno=0;fno<n;fno++)
 {if(frames[fno].pno==pno)
    return fno;
  }
 return -1;
}

int get_free_frame()
{int fno;
 for (fno=0; fno<n; fno++)
 {
  if (frames[fno].pno==-1)
   return(fno);
  }
 return(-1);
}

int get_mru_Frame()
{
 int lrufno=0,fno;
```

```
 for (fno=1; fno<n; fno++)
 {
  if(frames[fno].counter>frames[lrufno].counter)
   lrufno=fno;
 }
 return lrufno;
}
void main()
{
 int p_request[]={8,5,7,8,5,7,2,3,7,3,5,9,4,6,2};
 int size=sizeof(p_request)/2,currtime;
 int page_falts=0,i,j,fno;
 clrscr();
 printf("\nHow many frames:");  scanf("%d",&n);
 //initialize frames
 for (i=0; i<n; i++)
 { frames[i].pno=-1;
 }

 printf("\nPageNo     Page Frames     Page Fault");
 printf("\n-------------------------------------------------");
 currtime=0;
 for(i=0;i<size;i++)
 {
  j=page_found(p_request[i]);
  if(j==-1)  //page fault occurs
  {
   j=get_free_frame();
   if (j==-1) //no free frame - do replacement
    j=get_mru_Frame();
   page_falts++;
```

```
  frames[j].pno=p_request[i];
  frames[j].counter=currtime;
  printf("\n%4d\t ",p_request[i]);
  for (fno=0; fno<n; fno++)
   printf("%4d:%2d",frames[fno].pno,frames[fno].counter);
  printf(" : YES");
 }
 else//page found
 {frames[j].counter=currtime;
  printf("\n%4d\t ",p_request[i]);
  for (fno=0; fno<n; fno++)
   printf("%4d:%2d",frames[fno].pno,frames[fno].counter);
  printf(" : NO");

 }//else
 currtime++;
}//for
printf("\n----------------------------------------");
printf("\n Number of Page_Falts=%d",page_falts);
// getch();
}
```

2)

```
#include <stdio.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <dirent.h>
//#include <bfd.h>
```

```
char comm[100],*ptr;
char *args[10];
int tot_arg;

int search(char option, char *pattern, char *fname);

main()
{
	do
	{
		printf("myshell $ ");
		getcomm();
		sep_arg();
		take_action();
	}while(1);
} // main

sep_arg()
{
	int i,j;
	i = j = tot_arg = 0;

	args[tot_arg] = (char*)malloc(sizeof(char) * 20);
	for(i=0; comm[i] !='\0' ; i++)
	{
		if( comm[i] == ' ')
		{
			args[tot_arg][j] = '\0';
			tot_arg++;
			args[tot_arg] = (char*)malloc(sizeof(char) * 20);
			j=0;
```

```
                }
                else
                {
                        args[tot_arg][j] =comm[i];
                        j++;
                }
        } // while
        args[tot_arg][j] = '\0'; // complete last word
        return;
}

getcomm()
{
        int len;
        ptr = fgets(comm,80,stdin);
        len = strlen(comm);
        comm[len-1] = '\0';
        return;
} // getcomm

take_action()
{
        char str[100];
        pid_t   pid;
        int status;

        if(strcmp(args[0],"search") == 0 )
                search(*args[1],args[2],args[3]);
        else
        {
                if( ptr == NULL )
```

```
                exit(0);
        if ( strlen(comm) <= 1 )
                return;
        if( ( pid = fork() ) < 0 )
        {
                printf("\nUnable to create process");
        }
        else
        if(  pid == 0 ) // child
        {   sscanf(comm,"%s",comm);
                execvp(comm,args);
                strcpy(str,"./");
                strcat(str,args[0]);
                execvp(str,args);
                printf("\n%s : command not found",comm);
                exit(0);
        }
        else //parent
         waitpid(pid,&status,0);
    } // else
}// take action


int search(char option, char *pattern, char *fname)
{
    char buff[100],*ptr;
    int cnt;
    FILE *fp;
    int i;
    fp= fopen(fname,"r");
    if( fp == NULL )
```

```
{
        printf("\nUnable to open file");
        exit(0);
}
switch(option)
{
   case 'f' : //print first occ
                while ( !feof (fp))
                {
                        buff[0] = '\0';
                        fgets(buff,80,fp);
                        ptr = strstr(buff,pattern);
                        if( ptr != NULL )
                        {
                                printf("%s",buff);
                                break;
                        }
                }
                fclose(fp);
                break;
case 'a' : // print all occ
                while ( !feof (fp))
                {
                        buff[0] = '\0';
                        fgets(buff,80,fp);
                        ptr = strstr(buff,pattern);
                        if( ptr != NULL )
                                printf("%s",buff);
                }
                fclose(fp);
                break;
```

```
        } // switch
        return;
}//search
```

Slip7

1)

2)

```
#include <stdio.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <dirent.h>
//#include <bfd.h>
char comm[100],*ptr;
char *args[10];
int tot_arg;

int search(char option, char *pattern, char *fname);

main()
{
        do
        {
                printf("myshell $ ");
                getcomm();
                sep_arg();
                take_action();
```

```
        }while(1);
} // main

sep_arg()
{
        int i,j;
        i = j = tot_arg = 0;

        args[tot_arg] = (char*)malloc(sizeof(char) * 20);
        for(i=0; comm[i] !='\0' ; i++)
        {
                if( comm[i] == ' ')
                {
                        args[tot_arg][j] = '\0';
                        tot_arg++;
                        args[tot_arg] = (char*)malloc(sizeof(char) * 20);
                        j=0;
                }
                else
                {
                        args[tot_arg][j] =comm[i];
                        j++;
                }
        } // while
        args[tot_arg][j] = '\0'; // complete last word
        return;
}

getcomm()
{
        int len;
```

```
        ptr = fgets(comm,80,stdin);
        len = strlen(comm);
        comm[len-1] = '\0';
        return;
} // getcomm

take_action()
{
        char str[100];
        pid_t   pid;
        int status;

        if(strcmp(args[0],"search") == 0 )
                search(*args[1],args[2],args[3]);
        else
        {
                if( ptr == NULL )
                        exit(0);
                if ( strlen(comm) <= 1 )
                        return;
                if( ( pid = fork() ) < 0 )
                {
                        printf("\nUnable to create process");
                }
                else
                if(  pid == 0 ) // child
                {   sscanf(comm,"%s",comm);
                        execvp(comm,args);
                        strcpy(str,"./");
                        strcat(str,args[0]);
                        execvp(str,args);
```

```
                        printf("\n%s : command not found",comm);
                        exit(0);
                }
                else //parent
                 waitpid(pid,&status,0);
        } // else
}// take action



int search(char option, char *pattern, char *fname)
{
        char buff[100],*ptr;
        int cnt;
        FILE *fp;
        int i;
        fp= fopen(fname,"r");
        if( fp == NULL )
        {
                printf("\nUnable to open file");
                exit(0);
        }
        switch(option)
        {
           case 'a' : // print all occ
                        while ( !feof (fp))
                        {
                                buff[0] = '\0';
                                fgets(buff,80,fp);
                                ptr = strstr(buff,pattern);
                                if( ptr != NULL )
                                        printf("%s",buff);
```

```
                        }
                        fclose(fp);
                        break;
            case 'c' : // count occ
                        cnt=0;
                        while ( !feof (fp))
                        {
                                buff[0] = '\0';
                                fgets(buff,80,fp);
                                ptr = strstr(buff,pattern);
                                while(ptr != NULL )
                                {
                                        cnt++;
                                        ptr = ptr + strlen(pattern);
                                        ptr = strstr(ptr,pattern);
                                } // while
                        }
                        fclose(fp);
                        printf("\nThe serach string %s occurs %d times",pattern,cnt);
                        break;
        } // switch
        return;
}//search
```

Slip8

1)

```
/*Program for LRU Page replacement simulation*/
#include<stdio.h>
struct node
{
  int pno,reftime;
```

```
}frames[20];
int n;

int page_found(int pno)
{
  int fno;
  for(fno=0;fno<n;fno++)
            if(frames[fno].pno==pno)
              return fno;
            return -1;
}

int get_free_frame()
{  int fno;
  for (fno=0; fno<=n; fno++)
            if (frames[fno].pno==-1)
                        return(fno);
   return(-1);
}

int get_lru_frame()
{
  int fno;
  int selfno=0;
  for (fno=1; fno<n; fno++)
            if(frames[fno].reftime<frames[selfno].reftime)
            selfno=fno;
  return selfno;
}
main()
{
```

```
int p_request[]={8,5,7,8,5,7,2,3,7,3,5,9,4,6,2};
int size=15,currtime;
int page_falts=0,i,j,fno;

printf("\nHow many frames:");  scanf("%d",&n);
//initialize frames
for (i=0; i<n; i++)
{ frames[i].pno=-1;
        frames[i].reftime=-1;
}

printf("\nPageNo     Page Frames         Page Fault");
printf("\n-----------------------------------------");
currtime=0;
for(i=0;i<size;i++)
{
        j=page_found(p_request[i]);
        if(j==-1)  //page fault occurs
        {
          j=get_free_frame();
          if (j==-1) //no free frame - do page replacement
                j=get_lru_frame();
          page_falts++;
          frames[j].pno=p_request[i];
          frames[j].reftime=currtime;
          printf("\n%4d\t ",p_request[i]);
          for (fno=0; fno<n; fno++)
                printf("%4d",frames[fno].pno);
          printf(" : YES");
        }
       else//page found
```

```
            {
              printf("\n%4d\t ",p_request[i]);
              frames[j].reftime=currtime;
              for (fno=0; fno<n; fno++)
                        printf("%4d",frames[fno].pno);
              printf(" : NO");
            }
              currtime++;
   }
  printf("\n----------------------------------------");
  printf("\n Number of Page_Falts=%d",page_falts);

}
```

2)

```
#include <stdio.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <dirent.h>
//#include <bfd.h>
char comm[100],*ptr;
char *args[10];
int tot_arg;

int search(char option, char *pattern, char *fname);

main()
{
```

```
	do
	{
		printf("myshell $ ");
		getcomm();
		sep_arg();
		take_action();
	}while(1);
} // main

sep_arg()
{
	int i,j;
	i = j = tot_arg = 0;

	args[tot_arg] = (char*)malloc(sizeof(char) * 20);
	for(i=0; comm[i] !='\0' ; i++)
	{
		if( comm[i] == ' ')
		{
			args[tot_arg][j] = '\0';
			tot_arg++;
			args[tot_arg] = (char*)malloc(sizeof(char) * 20);
			j=0;
		}
		else
		{
			args[tot_arg][j] =comm[i];
			j++;
		}
	} // while
	args[tot_arg][j] = '\0'; // complete last word
```

```
        return;
}

getcomm()
{
        int len;
        ptr = fgets(comm,80,stdin);
        len = strlen(comm);
        comm[len-1] = '\0';
        return;
} // getcomm

take_action()
{
        char str[100];
        pid_t   pid;
        int status;

        if(strcmp(args[0],"search") == 0 )
                search(*args[1],args[2],args[3]);
        else
        {
                if( ptr == NULL )
                        exit(0);
                if ( strlen(comm) <= 1 )
                        return;
                if( ( pid = fork() ) < 0 )
                {
                        printf("\nUnable to create process");
                }
                else
```

```
                if(  pid == 0 ) // child
                {   sscanf(comm,"%s",comm);
                        execvp(comm,args);
                        strcpy(str,"./");
                        strcat(str,args[0]);
                        execvp(str,args);
                        printf("\n%s : command not found",comm);
                        exit(0);
                }
                else //parent
                 waitpid(pid,&status,0);
        } // else
}// take action

int search(char option, char *pattern, char *fname)
{
        char buff[100],*ptr;
        int cnt;
        FILE *fp;
        int i;
        fp= fopen(fname,"r");
        if( fp == NULL )
        {
                printf("\nUnable to open file");
                exit(0);
        }
        switch(option)
        {
           case 'f' : //print first occ
                        while ( !feof (fp))
                        {
```

```
                buff[0] = '\0';
                fgets(buff,80,fp);
                ptr = strstr(buff,pattern);
                if( ptr != NULL )
                {
                        printf("%s",buff);
                        break;
                }
        }
        fclose(fp);
        break;

case 'c' : // count occ
        cnt=0;
        while ( !feof (fp))
        {
                buff[0] = '\0';
                fgets(buff,80,fp);
                ptr = strstr(buff,pattern);
                while(ptr != NULL )
                {
                        cnt++;
                        ptr = ptr + strlen(pattern);
                        ptr = strstr(ptr,pattern);
                } // while
        }
        fclose(fp);
        printf("\nThe serach string %s occurs %d times",pattern,cnt);
        break;
    } // switch
    return;
```

```
}//search
```

Slip9

1)

```
/*Program for FIFO*/
#include<stdio.h>

int frame[3]={-1,-1,-1};
int pg_found(int pno)
{
	int i;
	for(i=0;i<3;i++)
		if(frame[i]==pno)
		return i;
	return -1;
}
int main()
{
	int p_req[]={8,5,7,8,5,7,2,3,7,3,5,9,4,6,2};
	int size=15,page_fault=0,i;

	printf("\n FIFO page replacement");
	printf("\n page no  -----------------page frames");
	printf("\n--------------------------------------");
	for(i=0;i<size;i++)
	{
		if(pg_found(p_req[i])==-1)
		{
			frame[page_fault%3]=p_req[i];
			page_fault++;
```

```
                        printf("\n %4d--------------------
%4d%4d%4d",p_req[i],frame[0],frame[1],frame[2]);
                }
                else
                        printf("\n %4d.......No Page fault",p_req[i]);
        }
        printf("\n-----------------------------------------");
        printf("\n Total page fault=%d",page_fault);

}
```

2)

```
#include <stdio.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <dirent.h>
//#include <bfd.h>
char comm[100],*ptr;
char *args[10];
int tot_arg;

int search(char option, char *pattern, char *fname);

main()
{
        do
        {
                printf("myshell $ ");
```

```
		getcomm();
		sep_arg();
		take_action();
	}while(1);
} // main

sep_arg()
{
	int i,j;
	i = j = tot_arg = 0;

	args[tot_arg] = (char*)malloc(sizeof(char) * 20);
	for(i=0; comm[i] !='\0' ; i++)
	{
		if( comm[i] == ' ')
		{
			args[tot_arg][j] = '\0';
			tot_arg++;
			args[tot_arg] = (char*)malloc(sizeof(char) * 20);
			j=0;
		}
		else
		{
			args[tot_arg][j] =comm[i];
			j++;
		}
	} // while
	args[tot_arg][j] = '\0'; // complete last word
	return;
}
```

```
getcomm()
{
        int len;
        ptr = fgets(comm,80,stdin);
        len = strlen(comm);
        comm[len-1] = '\0';
        return;
} // getcomm

take_action()
{
        char str[100];
        pid_t   pid;
        int status;

        if(strcmp(args[0],"search") == 0 )
                search(*args[1],args[2],args[3]);
        else
        {
                if( ptr == NULL )
                        exit(0);
                if ( strlen(comm) <= 1 )
                        return;
                if( ( pid = fork() ) < 0 )
                {
                        printf("\nUnable to create process");
                }
                else
                if(  pid == 0 ) // child
                {   sscanf(comm,"%s",comm);
                        execvp(comm,args);
```

```
                        strcpy(str,"./");
                        strcat(str,args[0]);
                        execvp(str,args);
                        printf("\n%s : command not found",comm);
                        exit(0);
                }
                else //parent
                 waitpid(pid,&status,0);
        } // else
}// take action

int search(char option, char *pattern, char *fname)
{
        char buff[100],*ptr;
        int cnt;
        FILE *fp;
        int i;
        fp= fopen(fname,"r");
        if( fp == NULL )
        {
                printf("\nUnable to open file");
                exit(0);
        }
        switch(option)
        {
           case 'f' : //print first occ
                        while ( !feof (fp))
                        {
                                buff[0] = '\0';
                                fgets(buff,80,fp);
                                ptr = strstr(buff,pattern);
```

```
                    if( ptr != NULL )
                    {
                        printf("%s",buff);
                        break;
                    }
                }
                fclose(fp);
                break;

    case 'a' : // print all occ
                while ( !feof (fp))
                {
                    buff[0] = '\0';
                    fgets(buff,80,fp);
                    ptr = strstr(buff,pattern);
                    if( ptr != NULL )
                        printf("%s",buff);
                }
                fclose(fp);
                break;
        } // switch
        return;
}//search
```

Slip10

1)

```
/*Program for FIFO*/
#include<stdio.h>

int frame[3]={-1,-1,-1};
int pg_found(int pno)
```

```
{
	int i;
	for(i=0;i<3;i++)
		if(frame[i]==pno)
		return i;
	return -1;
}
int main()
{
	int p_req[]={2,4,5,6,9,4,7,3,4,5,6,7,2,4,7,1};
	int size=15,page_fault=0,i;

	printf("\n FIFO page replacement");
	printf("\n page no  ----------------page frames");
	printf("\n--------------------------------------");
	for(i=0;i<size;i++)
	{
		if(pg_found(p_req[i])==-1)
		{
			frame[page_fault%3]=p_req[i];
			page_fault++;
			printf("\n %4d--------------------
%4d%4d%4d",p_req[i],frame[0],frame[1],frame[2]);
		}
		else
			printf("\n %4d.......No Page fault",p_req[i]);
	}
	printf("\n-----------------------------------------");
	printf("\n Total page fault=%d",page_fault);

}
```

2)

```
#include <stdio.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <dirent.h>
//#include <bfd.h>
char comm[100],*ptr;
char *args[10];
int tot_arg;

int list( char option, char dirname[]);

main()
{
	do
	{
		printf("myshell $ ");
		getcomm();
		sep_arg();
		take_action();
	}while(1);
} // main

sep_arg()
{
	int i,j;
	i = j = tot_arg = 0;
```

```
        args[tot_arg] = (char*)malloc(sizeof(char) * 20);
        for(i=0; comm[i] !='\0' ; i++)
        {
                if( comm[i] == ' ')
                {
                        args[tot_arg][j] = '\0';
                        tot_arg++;
                        args[tot_arg] = (char*)malloc(sizeof(char) * 20);
                        j=0;
                }
                else
                {
                        args[tot_arg][j] =comm[i];
                        j++;
                }
        } // while
        args[tot_arg][j] = '\0'; // complete last word
        return;
}

getcomm()
{
        int len;
        ptr = fgets(comm,80,stdin);
        len = strlen(comm);
        comm[len-1] = '\0';
        return;
} // getcomm

take_action()
```

```
{
        char str[100];
        pid_t   pid;
        int status;

        if(strcmp(args[0],"list") == 0 )
                list(*args[1],args[2]);
        else
        {
                if( ptr == NULL )
                        exit(0);
                if ( strlen(comm) <= 1 )
                        return;
                if( ( pid = fork() ) < 0 )
                {
                        printf("\nUnable to create process");
                }
                else
                if(  pid == 0 ) // child
                {   sscanf(comm,"%s",comm);
                        execvp(comm,args);
                        strcpy(str,"./");
                        strcat(str,args[0]);
                        execvp(str,args);
                        printf("\n%s : command not found",comm);
                        exit(0);
                }
                else //parent
                 waitpid(pid,&status,0);
        } // else
}// take action
```

```
int list( char option, char dirname[])
{
        DIR *dp;
        struct dirent *dent;
        struct stat buff;
        char orgdir[100];
        int cnt,reply;
        getcwd(orgdir,90);
        chdir(dirname);
        switch(option)
        {
          case 'f' : // list all dir entries
                        dp= opendir(".");
                        if( dp == NULL )
                        {
                                printf("\nUnable to open dir");
                                exit(0);
                        }
                        dent = readdir(dp);
                        while( dent != NULL )
                        {
                                printf("\n%s",dent->d_name);
                                dent= readdir(dp);
                        }
                        printf("\n");
                        break;

          case 'i' : // list all dir entries with inode
                        dp= opendir(".");
                        if( dp == NULL )
```

```
                    {
                        printf("\nUnable to open dir");
                        exit(0);
                    }
                    dent = readdir(dp);
                    while( dent != NULL )
                    {
                        reply = stat(dent->d_name,&buff);
                        if( reply < 0 )
                        printf("\nError");
                        else
                        printf("\n%s %ld",dent->d_name,buff.st_ino);
                        dent= readdir(dp);
                    }
                    closedir(dp);
                    break;
        default :   printf("\nInvalid option");
                    exit(0);
    } // switch
    chdir(orgdir);
    return(0);
}// main
```

Slip11

1)

```
/*Program for LFU Page replacement simulation*/
#include<stdio.h>

struct node
{
  int pno,freq;
```

```
}frames[20];
int n;

int page_found(int pno)
{
  int fno;
  for(fno=0;fno<n;fno++)
            if(frames[fno].pno==pno)
              return fno;
            return -1;
}

int get_free_frame()
{  int fno;
  for (fno=0; fno<=n; fno++)
            if (frames[fno].pno==-1)
                        return(fno);
   return(-1);
}

int get_lfu_frame()
{
  int fno;
  int selfno=0;
  for (fno=1; fno<n; fno++)
            if(frames[fno].freq<frames[selfno].freq)
            selfno=fno;
  return selfno;
}
int main()
{
```

```
int p_request[]={3,4,5,6,3,4,7,3,4,5,6,7,2,4,6};
int size=12;
int page_falts=0,i,j,fno;

printf("\nHow many frames:");  scanf("%d",&n);
//initialize frames
for (i=0; i<n; i++)
{ frames[i].pno=-1;
        frames[i].freq=0;
}

printf("\nPageNo     Page Frames         Page Fault");
printf("\n-------------------------------------------------");
for(i=0;i<size;i++)
{
        j=page_found(p_request[i]);
        if(j==-1)  //page fault occurs
        {
          j=get_free_frame();
          if (j==-1) //no free frame - do page replacement
                j=get_lfu_frame();
          page_falts++;
          frames[j].pno=p_request[i];
          frames[j].freq=1;
          printf("\n%4d\t ",p_request[i]);
          for (fno=0; fno<n; fno++)
                printf("%4d:%2d",frames[fno].pno,frames[fno].freq);
          printf(" : YES");
        }
        else //page found in frame j
        {
```

```
            printf("\n%4d\t ",p_request[i]);
            frames[j].freq++;
            for (fno=0; fno<n; fno++)
                    printf("%4d:%2d",frames[fno].pno,frames[fno].freq);
            printf(" : NO");
        }
  }
 printf("\n-----------------------------------------------------");
 printf("\n Number of Page_Falts=%d",page_falts);
// getch();
}
```

2)

```
#include <stdio.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <dirent.h>
//#include <bfd.h>
char comm[100],*ptr;
char *args[10];
int tot_arg;

int list( char option, char dirname[]);

main()
{
        do
        {
```

```
		printf("myshell $ ");
		getcomm();
		sep_arg();
		take_action();
	}while(1);
} // main

sep_arg()
{
	int i,j;
	i = j = tot_arg = 0;

	args[tot_arg] = (char*)malloc(sizeof(char) * 20);
	for(i=0; comm[i] !='\0' ; i++)
	{
		if( comm[i] == ' ')
		{
			args[tot_arg][j] = '\0';
			tot_arg++;
			args[tot_arg] = (char*)malloc(sizeof(char) * 20);
			j=0;
		}
		else
		{
			args[tot_arg][j] =comm[i];
			j++;
		}
	} // while
	args[tot_arg][j] = '\0'; // complete last word
	return;
}
```

```
getcomm()
{
        int len;
        ptr = fgets(comm,80,stdin);
        len = strlen(comm);
        comm[len-1] = '\0';
        return;
} // getcomm

take_action()
{
        char str[100];
        pid_t   pid;
        int status;

        if(strcmp(args[0],"list") == 0 )
                list(*args[1],args[2]);
        else
        {
                if( ptr == NULL )
                        exit(0);
                if ( strlen(comm) <= 1 )
                        return;
                if( ( pid = fork() ) < 0 )
                {
                        printf("\nUnable to create process");
                }
                else
                if(  pid == 0 ) // child
                {   sscanf(comm,"%s",comm);
```

```
                        execvp(comm,args);
                        strcpy(str,"./");
                        strcat(str,args[0]);
                        execvp(str,args);
                        printf("\n%s : command not found",comm);
                        exit(0);
                }
                else //parent
                 waitpid(pid,&status,0);
        } // else
}// take action

int list( char option, char dirname[])
{
        DIR *dp;
        struct dirent *dent;
        struct stat buff;
        char orgdir[100];
        int cnt,reply;
        getcwd(orgdir,90);
        chdir(dirname);
        switch(option)
        {
           case 'f' : // list all dir entries
                        dp= opendir(".");
                        if( dp == NULL )
                        {
                                printf("\nUnable to open dir");
                                exit(0);
                        }
                        dent = readdir(dp);
```

```
                        while( dent != NULL )
                        {
                                printf("\n%s",dent->d_name);
                                dent= readdir(dp);
                        }
                        printf("\n");
                        break;
            case 'n' : // list number all dir entries
                        cnt =0;
                        dp= opendir(".");
                        if( dp == NULL )
                        {
                                printf("\nUnable to open dir");
                                exit(0);
                        }
                        dent = readdir(dp);
                        while( dent != NULL )
                        {
                                cnt++;
                                dent= readdir(dp);
                        }
                        printf("\ntot dir entries = %d\n",cnt);
                        closedir(dp);
                        break;
            default :   printf("\nInvalid option");
                        exit(0);
        } // switch
        chdir(orgdir);
        return(0);
}// main
```

Slip12

1)

```
/*Program for LRU Page replacement simulation*/
#include<stdio.h>
struct node
{
  int pno,reftime;
}frames[20];
int n;

int page_found(int pno)
{
  int fno;
  for(fno=0;fno<n;fno++)
            if(frames[fno].pno==pno)
               return fno;
            return -1;
}

int get_free_frame()
{  int fno;
  for (fno=0; fno<=n; fno++)
            if (frames[fno].pno==-1)
                        return(fno);
   return(-1);
}

int get_lru_frame()
{
  int fno;
  int selfno=0;
```

```
  for (fno=1; fno<n; fno++)
            if(frames[fno].reftime<frames[selfno].reftime)
            selfno=fno;
  return selfno;
}
main()
{
   int p_request[]={3,4,5,6,3,4,7,3,4,5,6,7,2,4,6};
   int size=15,currtime;
   int page_falts=0,i,j,fno;

   printf("\nHow many frames:");  scanf("%d",&n);
   //initialize frames
   for (i=0; i<n; i++)
   { frames[i].pno=-1;
             frames[i].reftime=-1;
   }

   printf("\nPageNo     Page Frames         Page Fault");
   printf("\n-------------------------------------------");
   currtime=0;
   for(i=0;i<size;i++)
   {
             j=page_found(p_request[i]);
             if(j==-1)  //page fault occurs
             {
               j=get_free_frame();
               if (j==-1) //no free frame - do page replacement
                         j=get_lru_frame();
               page_falts++;
               frames[j].pno=p_request[i];
```

```
            frames[j].reftime=currtime;
            printf("\n%4d\t ",p_request[i]);
            for (fno=0; fno<n; fno++)
                    printf("%4d",frames[fno].pno);
            printf(" : YES");
           }
          else//page found
          {
            printf("\n%4d\t ",p_request[i]);
            frames[j].reftime=currtime;
            for (fno=0; fno<n; fno++)
                    printf("%4d",frames[fno].pno);
            printf(" : NO");
          }
            currtime++;
   }
  printf("\n----------------------------------------");
  printf("\n Number of Page_Falts=%d",page_falts);

}
```

2)

```
#include <stdio.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <dirent.h>
//#include <bfd.h>
char comm[100],*ptr;
```

```
char *args[10];
int tot_arg;

int list( char option, char dirname[]);

main()
{
	do
	{
		printf("myshell $ ");
		getcomm();
		sep_arg();
		take_action();
	}while(1);
} // main

sep_arg()
{
	int i,j;
	i = j = tot_arg = 0;

	args[tot_arg] = (char*)malloc(sizeof(char) * 20);
	for(i=0; comm[i] !='\0' ; i++)
	{
		if( comm[i] == ' ')
		{
			args[tot_arg][j] = '\0';
			tot_arg++;
			args[tot_arg] = (char*)malloc(sizeof(char) * 20);
			j=0;
		}
```

```
                else
                {
                        args[tot_arg][j] =comm[i];
                        j++;
                }
        } // while
        args[tot_arg][j] = '\0'; // complete last word
        return;
}

getcomm()
{
        int len;
        ptr = fgets(comm,80,stdin);
        len = strlen(comm);
        comm[len-1] = '\0';
        return;
} // getcomm

take_action()
{
        char str[100];
        pid_t   pid;
        int status;

        if(strcmp(args[0],"list") == 0 )
                list(*args[1],args[2]);
        else
        {
                if( ptr == NULL )
                        exit(0);
```

```
                if ( strlen(comm) <= 1 )
                        return;
                if( ( pid = fork() ) < 0 )
                {
                        printf("\nUnable to create process");
                }
                else
                if(  pid == 0 ) // child
                {   sscanf(comm,"%s",comm);
                        execvp(comm,args);
                        strcpy(str,"./");
                        strcat(str,args[0]);
                        execvp(str,args);
                        printf("\n%s : command not found",comm);
                        exit(0);
                }
                else //parent
                 waitpid(pid,&status,0);
        } // else
}// take action

int list( char option, char dirname[])
{
        DIR *dp;
        struct dirent *dent;
        struct stat buff;
        char orgdir[100];
        int cnt,reply;
        getcwd(orgdir,90);
        chdir(dirname);
        switch(option)
```

```
{
    case 'f' : // list all dir entries
                    dp= opendir(".");
                    if( dp == NULL )
                    {
                            printf("\nUnable to open dir");
                            exit(0);
                    }
                    dent = readdir(dp);
                    while( dent != NULL )
                    {
                            printf("\n%s",dent->d_name);
                            dent= readdir(dp);
                    }
                    printf("\n");
                    break;
    case 'n' : // list number all dir entries
                    cnt =0;
                    dp= opendir(".");
                    if( dp == NULL )
                    {
                            printf("\nUnable to open dir");
                            exit(0);
                    }
                    dent = readdir(dp);
                    while( dent != NULL )
                    {
                            cnt++;
                            dent= readdir(dp);
                    }
                    printf("\ntot dir entries = %d\n",cnt);
```

```
                        closedir(dp);
                        break;
            default :   printf("\nInvalid option");
                        exit(0);
        } // switch
        chdir(orgdir);
        return(0);
}// main
```

Slip13

1)

```
#include <stdio.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <dirent.h>
//#include <bfd.h>
char comm[100],*ptr;
char *args[10];
int tot_arg;

int typeline( char *option, char *fname);

main()
{
        do
        {
                printf("myshell $ ");
                getcomm();
```

```
                sep_arg();
                take_action();
        }while(1);
} // main

sep_arg()
{
        int i,j;
        i = j = tot_arg = 0;

        args[tot_arg] = (char*)malloc(sizeof(char) * 20);
        for(i=0; comm[i] !='\0' ; i++)
        {
                if( comm[i] == ' ')
                {
                        args[tot_arg][j] = '\0';
                        tot_arg++;
                        args[tot_arg] = (char*)malloc(sizeof(char) * 20);
                        j=0;
                }
                else
                {
                        args[tot_arg][j] =comm[i];
                        j++;
                }
        } // while
        args[tot_arg][j] = '\0'; // complete last word
        return;
}

getcomm()
```

```
{
        int len;
        ptr = fgets(comm,80,stdin);
        len = strlen(comm);
        comm[len-1] = '\0';
        return;
} // getcomm

take_action()
{
        char str[100];
        pid_t   pid;
        int status;

        if(strcmp(args[0],"search") == 0 )
                search(*args[1],args[2],args[3]);
        else
        if(strcmp(args[0],"count") == 0 )
                count(*args[1],args[2]);
        else
        if(strcmp(args[0],"typeline") == 0 )
                typeline(args[1],args[2]);
        else
        if(strcmp(args[0],"list") == 0 )
                list(*args[1],args[2]);
        else
        {
                if( ptr == NULL )
                        exit(0);
                if ( strlen(comm) <= 1 )
                        return;
```

```
                if( ( pid = fork() ) < 0 )
                {
                        printf("\nUnable to create process");
                }
                else
                if(  pid == 0 ) // child
                {   sscanf(comm,"%s",comm);
                        execvp(comm,args);
                        strcpy(str,"./");
                        strcat(str,args[0]);
                        execvp(str,args);
                        printf("\n%s : command not found",comm);
                        exit(0);
                }
                else //parent
                 waitpid(pid,&status,0);
        } // else
}// take action

int typeline( char *option, char *fname)
{
        FILE *fp;
        int tot_lines,ch,n,cnt;
        //---- count the lines
        fp= fopen(fname,"r");
        if( fp == NULL )
        {
                printf("\nUnable to open file");
                exit(0);
        }
        tot_lines = 0;//count total lines
```

```
        ch = fgetc(fp);
        while ( ch != EOF )
        {
                if( ch == 10 ) //new line char
                tot_lines++;
                ch = fgetc(fp);
        }
        fclose(fp);
        //---- take action
        if( strcmp(option,"a")== 0 ) // print all lines
        {
                fp= fopen(fname,"r");
                ch = fgetc(fp);
                while ( ch != EOF )
                {
                        printf("%c",ch);
                        ch = fgetc(fp);
                }
                fclose(fp);
        }
        else
        {
                printf("\nInvalid option");
                exit(0);
        }
        return(0);
} //typeline
```

2)

//simulation of Round Robin CPU Scheduling Algorithm

```
#include <stdio.h>
#include <stdlib.h>
#define TQ 2

struct proc_info
{
  int atime;
  int cpub;
} proc[20];

struct rqnode
 { int pid;
   struct rqnode *link;
 } *first=NULL, *curr, *prev, *last;

struct ganttchart
{
  int stime;
  int pid;
  int etime;
} gchart[20];

int i,n,ctime,gi=0,wtime[20],ttime[20],exectime;

void getprocs();
void start();
void addprocq();
void attachtoq(int );
void addgchart(int);
void dispgchart();
void disptimes();
```

```
int finished();
int getfirstproc();



 main()
{

  getprocs();
  ctime=0; //set current time to 0
  start();
  dispgchart();
  disptimes();
}

void getprocs()
{ printf("\nHow many processes: "); scanf("%d",&n);
  printf("\nPID\tATIME\tCPUB\n");
  for (i=1; i<=n; i++)
  { printf("%d\t",i);
    scanf("%d%d",&proc[i].atime,&proc[i].cpub);
   }
}//get_proc

void start()
{ int pid;
  addprocq(); //add procs in rq arrived at ctime
  pid=getfirstproc();//get short process
  while(!finished()) //while not finished all processes
  {
    if(pid!=-1)
    { //execute process pid
```

```
      ctime++;  //increment current time
      addprocq();
      proc[pid].cpub--;
      exectime++;
      if(proc[pid].cpub==0)//proc finished
      {
            printf("\nProc %d completed at time %d..",pid,ctime);
            pid=getfirstproc();//get short process
      }//if
    }//if
   else
   { ctime++;//no process in rq
    addprocq(); //add procs in rq arrived at ctime
   }
   //check if exectime of pid=TQ.
   //if yes, preempt current process, schedule new process
   if(first!=NULL && exectime==TQ)
   { attachtoq(pid);
     pid=getfirstproc();
   }
  }//while
  gchart[gi].etime=ctime;
}//start

void addprocq()
{
  for(i=1;i<=n;i++)
  { if(proc[i].atime==ctime)
      attachtoq(i);
   }//for
}//addprocq
```

```
void attachtoq(int pid)
{
  curr=(struct rqnode *)malloc(sizeof(struct rqnode));
  curr->pid=pid;curr->link=NULL;
  if(first==NULL)
    first=curr;
  else
    last->link=curr;
  last=curr;
}

int finished() //check all proc finished cpub
{
  for(i=1;i<=n;i++)
  { if(proc[i].cpub!=0)
      return(0);
  }
  return(1);
}


int getfirstproc()
{
  int pid;
  if(first==NULL)
    return(-1);
  pid=first->pid;
  curr=first; //remove from rq
  first=first->link;
  free(curr);
```

```
  addgchart(pid);
  exectime=0;
  return(pid);

}//getfirstproc



void addgchart(int pid)
{ static int ppid=-1;
  if(pid!=ppid)
  {
   gchart[++gi].pid=pid;
   gchart[gi].stime=ctime;
   gchart[gi-1].etime=gchart[gi].stime;
   ppid=pid;
  }
}

void dispgchart()
{ printf("\n");

  for(i=1;i<=gi;i++)
    printf("|----");
  printf("|\n");

  for(i=1;i<=gi;i++)
    printf("| %d  ",gchart[i].pid);
  printf("|\n");

  for(i=1;i<=gi;i++)
    printf("|----");
```

```
  printf("|\n");

  for(i=1;i<=gi;i++)
   printf("%d    ",gchart[i].stime);

  printf("%d\n",gchart[gi].etime);



}

void disptimes()
{ int sumwt=0,sumtt=0,pid;

  for(i=1;i<=gi;i++) //calculate turnaround time
 { pid=gchart[i].pid;
   ttime[pid]=(gchart[i].etime-proc[pid].atime);
 }
 for(i=1;i<=gi;i++) //calculate waiting time
 {
   pid=gchart[i].pid;
   wtime[pid]+=(gchart[i].stime-proc[pid].atime);
   proc[pid].atime=gchart[i].etime;
 }

  printf("\n**Waiting Time**");
  printf("\nPID\tWtime");
  for(i=1;i<=n;i++)
  {
    printf("\n%d\t%d",i,wtime[i]);
    sumwt+=wtime[i];
  }
```

```
  printf("\nAverage:%.2f",(float)sumwt/n);

  printf("\n**Turnaround Time**");
  printf("\nPID\t ttime");
  for(i=1;i<=n;i++)
  {
    printf("\n%d\t%d",i,ttime[i]);
    sumtt+=ttime[i];
  }
  printf("\nAverage:%.2f",(float)sumtt/n);

}
```

Slip14

1)

```
#include <stdio.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <dirent.h>
//#include <bfd.h>
char comm[100],*ptr;
char *args[10];
int tot_arg;

int typeline( char *option, char *fname);

main()
{
```

```
	do
	{
		printf("myshell $ ");
		getcomm();
		sep_arg();
		take_action();
	}while(1);
} // main

sep_arg()
{
	int i,j;
	i = j = tot_arg = 0;

	args[tot_arg] = (char*)malloc(sizeof(char) * 20);
	for(i=0; comm[i] !='\0' ; i++)
	{
		if( comm[i] == ' ')
		{
			args[tot_arg][j] = '\0';
			tot_arg++;
			args[tot_arg] = (char*)malloc(sizeof(char) * 20);
			j=0;
		}
		else
		{
			args[tot_arg][j] =comm[i];
			j++;
		}
	} // while
	args[tot_arg][j] = '\0'; // complete last word
```

```
        return;
}

getcomm()
{
        int len;
        ptr = fgets(comm,80,stdin);
        len = strlen(comm);
        comm[len-1] = '\0';
        return;
} // getcomm

take_action()
{
        char str[100];
        pid_t   pid;
        int status;

        if(strcmp(args[0],"typeline") == 0 )
                typeline(args[1],args[2]);
        else
        {
                if( ptr == NULL )
                        exit(0);
                if ( strlen(comm) <= 1 )
                        return;
                if( ( pid = fork() ) < 0 )
                {
                        printf("\nUnable to create process");
                }
                else
```

```
                if(  pid == 0 ) // child
                {   sscanf(comm,"%s",comm);
                        execvp(comm,args);
                        strcpy(str,"./");
                        strcat(str,args[0]);
                        execvp(str,args);
                        printf("\n%s : command not found",comm);
                        exit(0);
                }
                else //parent
                 waitpid(pid,&status,0);
        } // else
}// take action

int typeline( char *option, char *fname)
{
        FILE *fp;
        int tot_lines,ch,n,cnt;
        //---- count the lines
        fp= fopen(fname,"r");
        if( fp == NULL )
        {
                printf("\nUnable to open file");
                exit(0);
        }
        tot_lines = 0;//count total lines
        ch = fgetc(fp);
        while ( ch != EOF )
        {
                if( ch == 10 ) //new line char
                tot_lines++;
```

```
        ch = fgetc(fp);
}
fclose(fp);
//---- take action
if( strcmp(option,"a")== 0 ) // print all lines
{
        fp= fopen(fname,"r");
        ch = fgetc(fp);
        while ( ch != EOF )
        {
                printf("%c",ch);
                ch = fgetc(fp);
        }
        fclose(fp);
}
else
if( option[0] =='+') //print first n lines)
{
        n = atoi(option+1);
        if( n > tot_lines)
        {
                printf("\nInvalid  option");
                exit(0);
        }
        fp= fopen(fname,"r");
        cnt = 0;
        while( cnt < n )
        {
                ch = fgetc(fp);
                printf("%c",ch);
                if( ch == 10)
```

```
                                        cnt++;
                        }
                        fclose(fp);
                }
                else
                {
                        printf("\nInvalid option");
                        exit(0);
                }
                return(0);
} //typeline
```

2)

```
#include <stdio.h>

#include <stdlib.h>

struct proc_info
{
  int cpub;
  int atime;
  int flag;
} proc[20];

struct rqnode
 { int pid;
   struct rqnode *link;
 } *first=NULL, *curr, *prev, *last;

struct ganttchart
```

```
{
  int stime;
  int pid;
  int etime;
} gchart[20];

int i,n,ctime,gi=0,wtime[20]={0},ttime[20]={0};

void getprocs();
void start();
void addprocq();
void attachtoq(int);
void addgchart(int);
void dispgchart();
void calc_times();
void disptimes();
int finished();
int getfirstproc();
int getetime(int pid);


 main()
{
//clrscr();
  getprocs();
  ctime=0; //set current time to 0
  //randomize();
  start();
  dispgchart();
  disptimes();
}
```

```
void getprocs()
{ printf("\nHow many processes: "); scanf("%d",&n);
  printf("\nPID\tCPUB\tA.TIME\n");
  for (i=1; i<=n; i++)
  { printf("%d\t",i);
    scanf("%d%d",&proc[i].cpub,&proc[i].atime);
    proc[i].flag=0;
   }

}//get_proc

void start()
{ int pid;
  addprocq(); //add procs in rq arrived at ctime
  pid=getfirstproc();//get FCFS process
  while(!finished()) //while not finished all processes
  {
   if(pid!=-1)
   {
    //execute process pid
    addgchart(pid);
    ctime++;  //increment current time
    proc[pid].cpub--;
    if(proc[pid].cpub==0)//proc finished
    {
     //find turn around time
     ttime[pid]+=(ctime-proc[pid].atime);
         pid=getfirstproc();//get FCFS process
    }//if*/
   }//if
```

```
   else
   {
    ctime++;//no process in rq
    pid=getfirstproc();//get FCFS process
   }
   addprocq(); //add procs in rq arrived at ctime
  }//while
  gchart[gi].etime=ctime;
}//start

void addprocq()
{
  for(i=1;i<=n;i++)
  { if(proc[i].atime==ctime)
      attachtoq(i);
   }//for
}//addprocq

void attachtoq(int pid)
{
 curr=(struct rqnode *)malloc(sizeof(struct rqnode));
 curr->pid=pid;curr->link=NULL;
 if(first==NULL)
   first=curr;
 else
   last->link=curr;
 last=curr;
}

int finished() //check all proc finished cpub
{
```

```
  for(i=1;i<=n;i++)
  { if(proc[i].cpub!=0)
     return(0);
  }
  return(1);
}



int getfirstproc()
{
  int pid;
  if(first==NULL)
    return(-1);
  pid=first->pid;
  curr=first; //remove from rq
  first=first->link;
  free(curr);
  return(pid);

}//getfirstproc



void addgchart(int pid)
{static int ppid=-1;
 if(ppid!=pid)
 {
  gchart[++gi].pid=pid;
  gchart[gi].stime=ctime;
  gchart[gi-1].etime=gchart[gi].stime;
  //find waiting time of pid
  wtime[pid]+=(ctime-getetime(pid));
```

```
  ppid=pid;
 }
}

void dispgchart()
{ printf("\n");
  for(i=1;i<=gi;i++)
    printf("|----");
  printf("|\n");

  for(i=1;i<=gi;i++)
    printf("| %d  ",gchart[i].pid);
  printf("|\n");

  for(i=1;i<=gi;i++)
    printf("|----");
  printf("|\n");

  for(i=1;i<=gi;i++)
   printf("%d    ",gchart[i].stime);
  printf("%d\n",gchart[gi].etime);
  //getch();

}

int getetime(int pid)
{
  for(i=gi-1;i>=1;i--)
   if(gchart[i].pid==pid)
      return(gchart[i].etime);
  return(proc[pid].atime);
```

```
}

void disptimes()
{ int sumwt=0,sumtt=0,pid;



  printf("\n**Waiting Time**");
  printf("\nPID\tWtime");
  for(i=1;i<=n;i++)
  {
    printf("\n%d\t%d",i,wtime[i]);
    sumwt+=wtime[i];
  }
  printf("\nAverage:%.2f",(float)sumwt/n);

  printf("\n**Turnaround Time**");
  printf("\nPID\t ttime");
  for(i=1;i<=n;i++)
  {
    printf("\n%d\t%d",i,ttime[i]);
    sumtt+=ttime[i];
  }
  printf("\nAverage:%.2f",(float)sumtt/n);

  //getch();
}
```

Slip15

1)

```
#include <stdio.h>
```

```
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <dirent.h>
//#include <bfd.h>
char comm[100],*ptr;
char *args[10];
int tot_arg;

int list( char option, char dirname[]);

main()
{
	do
	{
		printf("myshell $ ");
		getcomm();
		sep_arg();
		take_action();
	}while(1);
} // main

sep_arg()
{
	int i,j;
	i = j = tot_arg = 0;

	args[tot_arg] = (char*)malloc(sizeof(char) * 20);
	for(i=0; comm[i] !='\0' ; i++)
```

```
        {
                if( comm[i] == ' ')
                {
                        args[tot_arg][j] = '\0';
                        tot_arg++;
                        args[tot_arg] = (char*)malloc(sizeof(char) * 20);
                        j=0;
                }
                else
                {
                        args[tot_arg][j] =comm[i];
                        j++;
                }
        } // while
        args[tot_arg][j] = '\0'; // complete last word
        return;
}

getcomm()
{
        int len;
        ptr = fgets(comm,80,stdin);
        len = strlen(comm);
        comm[len-1] = '\0';
        return;
} // getcomm

take_action()
{
        char str[100];
        pid_t   pid;
```

```
        int status;

        if(strcmp(args[0],"list") == 0 )
                list(*args[1],args[2]);
        else
        {
                if( ptr == NULL )
                        exit(0);
                if ( strlen(comm) <= 1 )
                        return;
                if( ( pid = fork() ) < 0 )
                {
                        printf("\nUnable to create process");
                }
                else
                if(  pid == 0 ) // child
                {   sscanf(comm,"%s",comm);
                        execvp(comm,args);
                        strcpy(str,"./");
                        strcat(str,args[0]);
                        execvp(str,args);
                        printf("\n%s : command not found",comm);
                        exit(0);
                }
                else //parent
                 waitpid(pid,&status,0);
        } // else
}// take action

int list( char option, char dirname[])
{
```

```
        DIR *dp;
        struct dirent *dent;
        struct stat buff;
        char orgdir[100];
        int cnt,reply;
        getcwd(orgdir,90);
        chdir(dirname);
        switch(option)
        {
          case 'f' : // list all dir entries
                        dp= opendir(".");
                        if( dp == NULL )
                        {
                                printf("\nUnable to open dir");
                                exit(0);
                        }
                        dent = readdir(dp);
                        while( dent != NULL )
                        {
                                printf("\n%s",dent->d_name);
                                dent= readdir(dp);
                        }
                        printf("\n");
                        break;

          default :     printf("\nInvalid option");
                        exit(0);
        } // switch
        chdir(orgdir);
        return(0);
}// main
```

2)

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

struct process_info
{
   int atime;
   int cpub;
}proc[10];

struct gantt_chart
{
   int procno;
   int stime;
   int etime;
}gchart[30];

int i, n, currtime=0, gindex,p, ttime[10], wtime[10], prevp=-1;
void get_process_input();
find_proc_toexec();
void print_gantt_chart();
finished_procs();
void print_times();
void gen_next_cpub();
void cpub_exec_proc(int p);

void main()
{
  int etime;
```

```
  get_process_input();
  while (!finished_procs())
  {
    p=find_proc_toexec();
    if (p!=-1)
      cpub_exec_proc(p);
    else
      currtime++;
  }
  gchart[gindex++].etime=currtime;
  printf("After Excecution of First CPU burst:\n");
  print_gantt_chart();
  print_times();
  // getch();
  gen_next_cpub();
  prevp=-1;
  while(!finished_procs())
  {
    p=find_proc_toexec();
    if(p!=-1)
      cpub_exec_proc(p);
    else
      currtime++;
  }
  gchart[gindex++].etime=currtime;
  printf("\nAfter Excecution of Next CPU burst:\n");
  print_gantt_chart();
  print_times();
  //getch();
}
```

```
void get_process_input()
{
  int pno;
  printf("\nHow many processes : ");
  scanf("%d",&n);
  printf("Proc   Arrival   First CPU   \n");
  printf("No.    Time      Burst       \n");
  pno=0;
  for(i=0; i<n; i++)
  {
    printf("\nP%d",pno++);
    scanf("%d%d",&proc[i].atime,&proc[i].cpub);
  }
}

find_proc_toexec()
{
  int mincpub=11, selproc=-1;
  for (i=0; i<n; i++)
  {
    if(proc[i].cpub!=0 && proc[i].atime<=currtime && proc[i].cpub<mincpub)
    {
      mincpub=proc[i].cpub;
      selproc=i;
    }
  }
  return selproc;
}

void cpub_exec_proc(int p)
```

```
{
  if(prevp!=p)
  {
    if(prevp!=-1) gchart[gindex++].etime=currtime;
      wtime[p]+=(currtime-getetime(p));
    gchart[gindex].procno=p;
    gchart[gindex].stime=currtime;
    prevp=p;
  }
  currtime++;
  proc[p].cpub-=1;
  ttime[p]=ttime[p]+1;
  if(proc[p].cpub==0)
  {
    proc[p].atime=currtime+2;
    ttime[p]+=wtime[p];
  }
}

finished_procs()
{
  for(i=0; i<n; i++)
    if(proc[i].cpub!=0)
      return (0);
  return (1);
}

void gen_next_cpub()
{
  random();
  printf("\n");
```

```
  printf("\nProc.No.\tNext CPU Burst");
  for(i=0; i<n; i++)
  {
    proc[i].cpub=random()%10;
    printf("\nP%d\t%d",i,proc[i].cpub);
  }
}

getetime(int p)
{
  for(i=gindex-1; i>0; i--)
    if (gchart[i].procno==p)
      return(gchart[i].etime);
  return proc[p].atime;
}

void print_gantt_chart()
{
  printf("\n|----|----|----|----|----|----|----|----|----------\n");
  printf("|");
  for(i=0; i<gindex; i++)
    printf(" %d  |",gchart[i].procno);
  printf("\n|----|----|----|----|----|----|----|----|------------\n");
  printf("%d",gchart[0].stime);
  for(i=0; i<gindex; i++)
    printf("%5d",gchart[i].etime);
}

void print_times()
{
  int totwtime=0,totttime=0;
```

```
  printf("\nProc   Waiting   TurnAround\n");
  printf("No.    Time      Time       \n");
  for(i=0; i<n; i++)
  {
    printf("%d\t%d\t%d\n",i,wtime[i],ttime[i]);
    totwtime+=wtime[i];
    totttime+=ttime[i];
  }
  printf("\nAverage Waiting Time = %.2f",(float)totwtime/n);
  printf("\nAverage Turnaround Time = %.2f",(float)totttime/n);
}
```

Slip16

1)

```
#include <stdio.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <dirent.h>
//#include <bfd.h>
char comm[100],*ptr;
char *args[10];
int tot_arg;

int count( char option, char *fname);

main()
{
        do
```

```
	{
		printf("myshell $ ");
		getcomm();
		sep_arg();
		take_action();
	}while(1);
} // main

sep_arg()
{
	int i,j;
	i = j = tot_arg = 0;

	args[tot_arg] = (char*)malloc(sizeof(char) * 20);
	for(i=0; comm[i] !='\0' ; i++)
	{
		if( comm[i] == ' ')
		{
			args[tot_arg][j] = '\0';
			tot_arg++;
			args[tot_arg] = (char*)malloc(sizeof(char) * 20);
			j=0;
		}
		else
		{
			args[tot_arg][j] =comm[i];
			j++;
		}
	} // while
	args[tot_arg][j] = '\0'; // complete last word
	return;
```

```
}

getcomm()
{
        int len;
        ptr = fgets(comm,80,stdin);
        len = strlen(comm);
        comm[len-1] = '\0';
        return;
} // getcomm

take_action()
{
        char str[100];
        pid_t   pid;
        int status;

        if(strcmp(args[0],"count") == 0 )
                count(*args[1],args[2]);
        else
        {
                if( ptr == NULL )
                        exit(0);
                if ( strlen(comm) <= 1 )
                        return;
                if( ( pid = fork() ) < 0 )
                {
                        printf("\nUnable to create process");
                }
                else
                if(  pid == 0 ) // child
```

```
                {   sscanf(comm,"%s",comm);
                        execvp(comm,args);
                        strcpy(str,"./");
                        strcat(str,args[0]);
                        execvp(str,args);
                        printf("\n%s : command not found",comm);
                        exit(0);
                }
                else //parent
                 waitpid(pid,&status,0);
        } // else
}// take action

int count( char option, char *fname)// count c/w/l   <filename>
{
        int ccnt,wcnt,lcnt,wflag;
        int ch;
        FILE *fp;
        int i;
        fp= fopen(fname,"r");
        if( fp == NULL )
        {
                printf("\nUnable to open file");
                exit(0);
        }
        ccnt =wcnt = lcnt = 0;
        wflag=1;
        ch = fgetc(fp);
        while ( ch != EOF)
        {
                ccnt++;
```

```
        if( ch ==' ' || ch== '\t')
        {
                if( wflag == 0 )
                {
                        wcnt++;
                        wflag = 1;
                }
        }
        else
        if( ch == '\n')
        {
                lcnt++;
                if( wflag == 0 )
                {
                        wcnt++;
                        wflag = 1;
                }
        }
        else
        {
                wflag = 0;
        }
        ch = fgetc(fp);
}//while
if( wflag == 0 )
{
        wcnt++;
        lcnt++;
}
fclose(fp);
switch(option)
```

```
        {
        case 'c' : // print all chars
                        printf("\nTot chars = %d\n",ccnt);
                        break;
        case 'w' : // print all words
                        printf("\nTot words = %d\n",wcnt);
                        break;
        } // switch
        return;
}// count
```

2)

```
//simulation of NP Priority CPU Scheduling Algorithm

#include <stdio.h>

#include <stdlib.h>

struct proc_info
{
  int atime;
  int cpub;
  int prio;
} proc[20];

struct rqnode
 { int pid,prio;
   struct rqnode *link;
 } *first=NULL, *curr, *prev, *last;

struct ganttchart
```

```
{
  int stime;
  int pid;
  int etime;
} gchart[20];

int i,n,ctime,gi=0,wtime[20],ttime[20];

void getprocs();
void start();
void addprocq();
void attachtoq(int );
void addgchart(int);
void dispgchart();
void disptimes();

void main()
{ //clrscr();
  //randomize();
  getprocs();
  ctime=0; //set current time to 0
  start();
  dispgchart();
  disptimes();
}

void getprocs()
{ printf("\nHow many processes: "); scanf("%d",&n);
  printf("\nPID\tATIME\tCPUB\tPriority\n");
  for (i=1; i<=n; i++)
  { printf("%d\t",i);
```

```
    scanf("%d%d%d",&proc[i].atime,&proc[i].cpub,&proc[i].prio);
   }
}//get_proc

void start()
{ int pid;
  addprocq(); //add procs in rq arrived at ctime
  pid=getfirstproc();//get short process
  while(!finished()) //while not finished all processes
  {
    if(pid!=-1)
    { //execute process pid
      ctime++;  //increment current time
      proc[pid].cpub--;
      if(proc[pid].cpub==0)//proc finished
      {
            printf("\nProc %d completed at time %d..",pid,ctime);
            ttime[pid]=ctime-proc[pid].atime;
            pid=getfirstproc();//get high prio process
      }//if
    }//if
   else
    ctime++;//no process in rq

   addprocq(); //add procs in rq arrived at ctime
  }//while
  gchart[gi].etime=ctime;
}//start

void addprocq()
{
```

```
  for(i=1;i<=n;i++)
  { if(proc[i].atime==ctime)
      attachtoq(i);
   }//for
}//addprocq

void attachtoq(int pid)
{
  struct rqnode *selp,*pselp;
  curr=(struct rqnode *)malloc(sizeof(struct rqnode));
  curr->pid=pid;curr->prio=proc[pid].prio; curr->link=NULL;
  if(first==NULL)
  { first=curr;
    return;
  }
  selp=first;
  while(selp->prio<=curr->prio && selp!=NULL)
  { pselp=selp;
    selp=selp->link;
  }
  if(selp==first)
  { curr->link=first;
    first=curr;
  }
  else
  {pselp->link=curr;
   curr->link=selp;
   }
}

int finished() //check all proc finished cpub
```

```
{
  for(i=1;i<=n;i++)
  { if(proc[i].cpub!=0)
     return(0);
  }
  return(1);
}

int getfirstproc()
{
  int pid;
  if(first==NULL)
    return(-1);
  pid=first->pid;
  curr=first; //remove from rq
  first=first->link;
  free(curr);
  addgchart(pid);
  wtime[pid]=ctime-proc[pid].atime;
  return(pid);

}//getfirstproc



void addgchart(int pid)
{
   gchart[++gi].pid=pid;
   gchart[gi].stime=ctime;
   gchart[gi-1].etime=gchart[gi].stime;
}
```

```
void dispgchart()
{ printf("\n");

  for(i=1;i<=gi;i++)
    printf("|----");
  printf("|\n");

  for(i=1;i<=gi;i++)
    printf("| %d  ",gchart[i].pid);
  printf("|\n");

  for(i=1;i<=gi;i++)
    printf("|----");
  printf("|\n");

  for(i=1;i<=gi;i++)
   printf("%d    ",gchart[i].stime);

  printf("%d\n",gchart[gi].etime);
 // getch();

}

void disptimes()
{ int sumwt=0,sumtt=0,pid;

  printf("\n**Waiting Time**");
  printf("\nPID\tWtime");
  for(i=1;i<=n;i++)
  {
    printf("\n%d\t%d",i,wtime[i]);
```

```
    sumwt+=wtime[i];
  }
  printf("\nAverage:%.2f",(float)sumwt/n);

  printf("\n**Turnaround Time**");
  printf("\nPID\t ttime");
  for(i=1;i<=n;i++)
  {
    printf("\n%d\t%d",i,ttime[i]);
    sumtt+=ttime[i];
  }
  printf("\nAverage:%.2f",(float)sumtt/n);
  //getch();
}
```

Slip17

1)

2)

```
#include <stdio.h>
//#include <conio.h>
#include <stdlib.h>

struct proc_info
{
  int cpub;
  int atime;
  int flag;
} proc[20];
```

```
struct rqnode
 { int pid;
   struct rqnode *link;
 } *first=NULL, *curr, *prev, *last;

struct ganttchart
{
  int stime;
  int pid;
  int etime;
} gchart[20];

int i,n,ctime,gi=0,wtime[20]={0},ttime[20]={0};

void getprocs();
void start();
void addprocq();
void attachtoq(int);
void addgchart(int);
void dispgchart();
void calc_times();
void disptimes();
int finished();
int getfirstproc();
int getetime(int pid);


 main()
{
//clrscr();
  getprocs();
```

```
  ctime=0; //set current time to 0
  //randomize();
  start();
  dispgchart();
  disptimes();
}

void getprocs()
{ printf("\nHow many processes: "); scanf("%d",&n);
  printf("\nPID\tCPUB\tA.TIME\n");
  for (i=1; i<=n; i++)
  { printf("%d\t",i);
    scanf("%d%d",&proc[i].cpub,&proc[i].atime);
    proc[i].flag=0;
   }

}//get_proc

void start()
{ int pid;
  addprocq(); //add procs in rq arrived at ctime
  pid=getfirstproc();//get FCFS process
  while(!finished()) //while not finished all processes
  {
   if(pid!=-1)
   {
    //execute process pid
    addgchart(pid);
    ctime++;  //increment current time
    proc[pid].cpub--;
    if(proc[pid].cpub==0)//proc finished
```

```
    {
     //find turn around time
     ttime[pid]+=(ctime-proc[pid].atime);
         pid=getfirstproc();//get FCFS process
    }//if*/
   }//if
   else
   {
    ctime++;//no process in rq
    pid=getfirstproc();//get FCFS process
   }
   addprocq(); //add procs in rq arrived at ctime
  }//while
  gchart[gi].etime=ctime;
}//start

void addprocq()
{
  for(i=1;i<=n;i++)
  { if(proc[i].atime==ctime)
      attachtoq(i);
   }//for
}//addprocq

void attachtoq(int pid)
{
 curr=(struct rqnode *)malloc(sizeof(struct rqnode));
 curr->pid=pid;curr->link=NULL;
 if(first==NULL)
   first=curr;
 else
```

```
   last->link=curr;
 last=curr;
}

int finished() //check all proc finished cpub
{
  for(i=1;i<=n;i++)
  { if(proc[i].cpub!=0)
     return(0);
  }
  return(1);
}


int getfirstproc()
{
  int pid;
  if(first==NULL)
    return(-1);
  pid=first->pid;
  curr=first; //remove from rq
  first=first->link;
  free(curr);
  return(pid);

}//getfirstproc


void addgchart(int pid)
{static int ppid=-1;
 if(ppid!=pid)
```

```
 {
  gchart[++gi].pid=pid;
  gchart[gi].stime=ctime;
  gchart[gi-1].etime=gchart[gi].stime;
  //find waiting time of pid
  wtime[pid]+=(ctime-getetime(pid));
  ppid=pid;
 }
}

void dispgchart()
{ printf("\n");
  for(i=1;i<=gi;i++)
    printf("|----");
  printf("|\n");

  for(i=1;i<=gi;i++)
    printf("| %d  ",gchart[i].pid);
  printf("|\n");

  for(i=1;i<=gi;i++)
    printf("|----");
  printf("|\n");

  for(i=1;i<=gi;i++)
   printf("%d    ",gchart[i].stime);
  printf("%d\n",gchart[gi].etime);
  //getch();

}
```

```
int getetime(int pid)
{
  for(i=gi-1;i>=1;i--)
   if(gchart[i].pid==pid)
      return(gchart[i].etime);
  return(proc[pid].atime);

}

void disptimes()
{ int sumwt=0,sumtt=0,pid;



  printf("\n**Waiting Time**");
  printf("\nPID\tWtime");
  for(i=1;i<=n;i++)
  {
    printf("\n%d\t%d",i,wtime[i]);
    sumwt+=wtime[i];
  }
  printf("\nAverage:%.2f",(float)sumwt/n);

  printf("\n**Turnaround Time**");
  printf("\nPID\t ttime");
  for(i=1;i<=n;i++)
  {
    printf("\n%d\t%d",i,ttime[i]);
    sumtt+=ttime[i];
  }
  printf("\nAverage:%.2f",(float)sumtt/n);
```

```
}
```

Slip18

1)

```
/*Program for LRU Page replacement simulation*/
#include<stdio.h>
struct node
{
  int pno,reftime;
}frames[20];
int n;

int page_found(int pno)
{
  int fno;
  for(fno=0;fno<n;fno++)
            if(frames[fno].pno==pno)
               return fno;
            return -1;
}

int get_free_frame()
{  int fno;
  for (fno=0; fno<=n; fno++)
            if (frames[fno].pno==-1)
                        return(fno);
   return(-1);
}

int get_lru_frame()
```

```
{
  int fno;
  int selfno=0;
  for (fno=1; fno<n; fno++)
            if(frames[fno].reftime<frames[selfno].reftime)
            selfno=fno;
  return selfno;
}
main()
{
   int p_request[]={3,4,5,6,3,4,7,3,4,5,6,7,2,4,6};
   int size=15,currtime;
   int page_falts=0,i,j,fno;

   printf("\nHow many frames:");  scanf("%d",&n);
   //initialize frames
   for (i=0; i<n; i++)
   { frames[i].pno=-1;
             frames[i].reftime=-1;
   }

   printf("\nPageNo     Page Frames         Page Fault");
   printf("\n-------------------------------------------");
   currtime=0;
   for(i=0;i<size;i++)
   {
             j=page_found(p_request[i]);
             if(j==-1)  //page fault occurs
             {
               j=get_free_frame();
               if (j==-1) //no free frame - do page replacement
```

```
                j=get_lru_frame();
            page_falts++;
            frames[j].pno=p_request[i];
            frames[j].reftime=currtime;
            printf("\n%4d\t ",p_request[i]);
            for (fno=0; fno<n; fno++)
                printf("%4d",frames[fno].pno);
            printf(" : YES");
          }
         else//page found
         {
            printf("\n%4d\t ",p_request[i]);
            frames[j].reftime=currtime;
            for (fno=0; fno<n; fno++)
                printf("%4d",frames[fno].pno);
            printf(" : NO");
         }
            currtime++;
  }
 printf("\n----------------------------------------");
 printf("\n Number of Page_Falts=%d",page_falts);

}
```

2)

```
#include <stdio.h>
//#include <conio.h>
#include <stdlib.h>

struct proc_info
```

```
{
  int cpub;
  int atime;
  int flag;
} proc[20];

struct rqnode
 { int pid;
   struct rqnode *link;
 } *first=NULL, *curr, *prev, *last;

struct ganttchart
{
  int stime;
  int pid;
  int etime;
} gchart[20];

int i,n,ctime,gi=0,wtime[20]={0},ttime[20]={0};

void getprocs();
void start();
void addprocq();
void attachtoq(int);
void addgchart(int);
void dispgchart();
void calc_times();
void disptimes();
int finished();
int getfirstproc();
int getetime(int pid);
```

```
 main()
{
//clrscr();
  getprocs();
  ctime=0; //set current time to 0
  //randomize();
  start();
  dispgchart();
  disptimes();
}

void getprocs()
{ printf("\nHow many processes: "); scanf("%d",&n);
  printf("\nPID\tCPUB\tA.TIME\n");
  for (i=1; i<=n; i++)
  { printf("%d\t",i);
    scanf("%d%d",&proc[i].cpub,&proc[i].atime);
    proc[i].flag=0;
   }

}//get_proc

void start()
{ int pid;
  addprocq(); //add procs in rq arrived at ctime
  pid=getfirstproc();//get FCFS process
  while(!finished()) //while not finished all processes
  {
   if(pid!=-1)
```

```
  {
   //execute process pid
   addgchart(pid);
   ctime++;  //increment current time
   proc[pid].cpub--;
   if(proc[pid].cpub==0)//proc finished
   {
    //find turn around time
    ttime[pid]+=(ctime-proc[pid].atime);
        pid=getfirstproc();//get FCFS process
   }//if*/
  }//if
  else
  {
   ctime++;//no process in rq
   pid=getfirstproc();//get FCFS process
  }
  addprocq(); //add procs in rq arrived at ctime
 }//while
 gchart[gi].etime=ctime;
}//start

void addprocq()
{
 for(i=1;i<=n;i++)
 { if(proc[i].atime==ctime)
     attachtoq(i);
  }//for
}//addprocq

void attachtoq(int pid)
```

```
{
 curr=(struct rqnode *)malloc(sizeof(struct rqnode));
 curr->pid=pid;curr->link=NULL;
 if(first==NULL)
   first=curr;
 else
   last->link=curr;
 last=curr;
}

int finished() //check all proc finished cpub
{
  for(i=1;i<=n;i++)
  { if(proc[i].cpub!=0)
     return(0);
  }
  return(1);
}


int getfirstproc()
{
  int pid;
  if(first==NULL)
    return(-1);
  pid=first->pid;
  curr=first; //remove from rq
  first=first->link;
  free(curr);
  return(pid);
```

```
}//getfirstproc



void addgchart(int pid)
{static int ppid=-1;
 if(ppid!=pid)
 {
  gchart[++gi].pid=pid;
  gchart[gi].stime=ctime;
  gchart[gi-1].etime=gchart[gi].stime;
  //find waiting time of pid
  wtime[pid]+=(ctime-getetime(pid));
  ppid=pid;
 }
}

void dispgchart()
{ printf("\n");
  for(i=1;i<=gi;i++)
    printf("|----");
  printf("|\n");

  for(i=1;i<=gi;i++)
    printf("| %d  ",gchart[i].pid);
  printf("|\n");

  for(i=1;i<=gi;i++)
    printf("|----");
  printf("|\n");

  for(i=1;i<=gi;i++)
```

```
   printf("%d    ",gchart[i].stime);
  printf("%d\n",gchart[gi].etime);
  //getch();

}

int getetime(int pid)
{
  for(i=gi-1;i>=1;i--)
   if(gchart[i].pid==pid)
      return(gchart[i].etime);
  return(proc[pid].atime);

}

void disptimes()
{ int sumwt=0,sumtt=0,pid;



  printf("\n**Waiting Time**");
  printf("\nPID\tWtime");
  for(i=1;i<=n;i++)
  {
    printf("\n%d\t%d",i,wtime[i]);
    sumwt+=wtime[i];
  }
  printf("\nAverage:%.2f",(float)sumwt/n);

  printf("\n**Turnaround Time**");
  printf("\nPID\t ttime");
  for(i=1;i<=n;i++)
```

```
  {
    printf("\n%d\t%d",i,ttime[i]);
    sumtt+=ttime[i];
  }
  printf("\nAverage:%.2f",(float)sumtt/n);



}
```

Slip19

1)

```
#include <stdio.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <dirent.h>
//#include <bfd.h>
char comm[100],*ptr;
char *args[10];
int tot_arg;

int list( char option, char dirname[]);

main()
{
        do
        {
                printf("myshell $ ");
                getcomm();
```

```
                sep_arg();
                take_action();
        }while(1);
} // main

sep_arg()
{
        int i,j;
        i = j = tot_arg = 0;

        args[tot_arg] = (char*)malloc(sizeof(char) * 20);
        for(i=0; comm[i] !='\0' ; i++)
        {
                if( comm[i] == ' ')
                {
                        args[tot_arg][j] = '\0';
                        tot_arg++;
                        args[tot_arg] = (char*)malloc(sizeof(char) * 20);
                        j=0;
                }
                else
                {
                        args[tot_arg][j] =comm[i];
                        j++;
                }
        } // while
        args[tot_arg][j] = '\0'; // complete last word
        return;
}

getcomm()
```

```
{
        int len;
        ptr = fgets(comm,80,stdin);
        len = strlen(comm);
        comm[len-1] = '\0';
        return;
} // getcomm

take_action()
{
        char str[100];
        pid_t   pid;
        int status;

        if(strcmp(args[0],"list") == 0 )
                list(*args[1],args[2]);
        else
        {
                if( ptr == NULL )
                        exit(0);
                if ( strlen(comm) <= 1 )
                        return;
                if( ( pid = fork() ) < 0 )
                {
                        printf("\nUnable to create process");
                }
                else
                if(  pid == 0 ) // child
                {   sscanf(comm,"%s",comm);
                        execvp(comm,args);
                        strcpy(str,"./");
```

```
                        strcat(str,args[0]);
                        execvp(str,args);
                        printf("\n%s : command not found",comm);
                        exit(0);
                }
                else //parent
                 waitpid(pid,&status,0);
        } // else
}// take action

int list( char option, char dirname[])
{
        DIR *dp;
        struct dirent *dent;
        struct stat buff;
        char orgdir[100];
        int cnt,reply;
        getcwd(orgdir,90);
        chdir(dirname);
        switch(option)
        {
           case 'f' : // list all dir entries
                        dp= opendir(".");
                        if( dp == NULL )
                        {
                                printf("\nUnable to open dir");
                                exit(0);
                        }
                        dent = readdir(dp);
                        while( dent != NULL )
                        {
```

```
                    printf("\n%s",dent->d_name);
                    dent= readdir(dp);
                }
                printf("\n");
                break;

        default :   printf("\nInvalid option");
                    exit(0);
        } // switch
        chdir(orgdir);
        return(0);
}// main
```

2)

```
//simulation of Round Robin CPU Scheduling Algorithm

#include <stdio.h>
#include <stdlib.h>
#define TQ 2

struct proc_info
{
  int atime;
  int cpub;
} proc[20];

struct rqnode
 { int pid;
   struct rqnode *link;
 } *first=NULL, *curr, *prev, *last;
```

```
struct ganttchart
{
  int stime;
  int pid;
  int etime;
} gchart[20];

int i,n,ctime,gi=0,wtime[20],ttime[20],exectime;

void getprocs();
void start();
void addprocq();
void attachtoq(int );
void addgchart(int);
void dispgchart();
void disptimes();
int finished();
int getfirstproc();


 main()
{

  getprocs();
  ctime=0; //set current time to 0
  start();
  dispgchart();
  disptimes();
}

void getprocs()
```

```
{ printf("\nHow many processes: "); scanf("%d",&n);
  printf("\nPID\tATIME\tCPUB\n");
  for (i=1; i<=n; i++)
  { printf("%d\t",i);
    scanf("%d%d",&proc[i].atime,&proc[i].cpub);
   }
}//get_proc

void start()
{ int pid;
  addprocq(); //add procs in rq arrived at ctime
  pid=getfirstproc();//get short process
  while(!finished()) //while not finished all processes
  {
    if(pid!=-1)
    { //execute process pid
      ctime++;  //increment current time
      addprocq();
      proc[pid].cpub--;
      exectime++;
      if(proc[pid].cpub==0)//proc finished
      {
            printf("\nProc %d completed at time %d..",pid,ctime);
            pid=getfirstproc();//get short process
      }//if
    }//if
   else
   { ctime++;//no process in rq
    addprocq(); //add procs in rq arrived at ctime
   }
   //check if exectime of pid=TQ.
```

```
   //if yes, preempt current process, schedule new process
   if(first!=NULL && exectime==TQ)
   { attachtoq(pid);
     pid=getfirstproc();
   }
  }//while
  gchart[gi].etime=ctime;
}//start

void addprocq()
{
  for(i=1;i<=n;i++)
  { if(proc[i].atime==ctime)
      attachtoq(i);
   }//for
}//addprocq

void attachtoq(int pid)
{
  curr=(struct rqnode *)malloc(sizeof(struct rqnode));
  curr->pid=pid;curr->link=NULL;
  if(first==NULL)
    first=curr;
  else
    last->link=curr;
  last=curr;
}

int finished() //check all proc finished cpub
{
  for(i=1;i<=n;i++)
```

```
  { if(proc[i].cpub!=0)
     return(0);
  }
  return(1);
}


int getfirstproc()
{
  int pid;
  if(first==NULL)
    return(-1);
  pid=first->pid;
  curr=first; //remove from rq
  first=first->link;
  free(curr);
  addgchart(pid);
  exectime=0;
  return(pid);

}//getfirstproc


void addgchart(int pid)
{ static int ppid=-1;
  if(pid!=ppid)
  {
   gchart[++gi].pid=pid;
   gchart[gi].stime=ctime;
   gchart[gi-1].etime=gchart[gi].stime;
   ppid=pid;
```

```
  }
}

void dispgchart()
{ printf("\n");

  for(i=1;i<=gi;i++)
    printf("|----");
  printf("|\n");

  for(i=1;i<=gi;i++)
    printf("| %d  ",gchart[i].pid);
  printf("|\n");

  for(i=1;i<=gi;i++)
    printf("|----");
  printf("|\n");

  for(i=1;i<=gi;i++)
   printf("%d    ",gchart[i].stime);

  printf("%d\n",gchart[gi].etime);



}

void disptimes()
{ int sumwt=0,sumtt=0,pid;

  for(i=1;i<=gi;i++) //calculate turnaround time
 { pid=gchart[i].pid;
```

```
  ttime[pid]=(gchart[i].etime-proc[pid].atime);
 }
 for(i=1;i<=gi;i++) //calculate waiting time
 {
  pid=gchart[i].pid;
  wtime[pid]+=(gchart[i].stime-proc[pid].atime);
  proc[pid].atime=gchart[i].etime;
 }

 printf("\n**Waiting Time**");
 printf("\nPID\tWtime");
 for(i=1;i<=n;i++)
 {
   printf("\n%d\t%d",i,wtime[i]);
   sumwt+=wtime[i];
 }
 printf("\nAverage:%.2f",(float)sumwt/n);

 printf("\n**Turnaround Time**");
 printf("\nPID\t ttime");
 for(i=1;i<=n;i++)
 {
   printf("\n%d\t%d",i,ttime[i]);
   sumtt+=ttime[i];
 }
 printf("\nAverage:%.2f",(float)sumtt/n);

}
```

Slip20

1)

```
#include <stdio.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <dirent.h>
//#include <bfd.h>
char comm[100],*ptr;
char *args[10];
int tot_arg;

int typeline( char *option, char *fname);

main()
{
	do
	{
		printf("myshell $ ");
		getcomm();
		sep_arg();
		take_action();
	}while(1);
} // main

sep_arg()
{
	int i,j;
	i = j = tot_arg = 0;

	args[tot_arg] = (char*)malloc(sizeof(char) * 20);
```

```
	for(i=0; comm[i] !='\0' ; i++)
	{
		if( comm[i] == ' ')
		{
			args[tot_arg][j] = '\0';
			tot_arg++;
			args[tot_arg] = (char*)malloc(sizeof(char) * 20);
			j=0;
		}
		else
		{
			args[tot_arg][j] =comm[i];
			j++;
		}
	} // while
	args[tot_arg][j] = '\0'; // complete last word
	return;
}

getcomm()
{
	int len;
	ptr = fgets(comm,80,stdin);
	len = strlen(comm);
	comm[len-1] = '\0';
	return;
} // getcomm

take_action()
{
	char str[100];
```

```
        pid_t   pid;
        int status;

        if(strcmp(args[0],"typeline") == 0 )
                typeline(args[1],args[2]);
        else
        {
                if( ptr == NULL )
                        exit(0);
                if ( strlen(comm) <= 1 )
                        return;
                if( ( pid = fork() ) < 0 )
                {
                        printf("\nUnable to create process");
                }
                else
                if(  pid == 0 ) // child
                {   sscanf(comm,"%s",comm);
                        execvp(comm,args);
                        strcpy(str,"./");
                        strcat(str,args[0]);
                        execvp(str,args);
                        printf("\n%s : command not found",comm);
                        exit(0);
                }
                else //parent
                 waitpid(pid,&status,0);
        } // else
}// take action

int typeline( char *option, char *fname)
```

```
{
        FILE *fp;
        int tot_lines,ch,n,cnt;
        //---- count the lines
        fp= fopen(fname,"r");
        if( fp == NULL )
        {
                printf("\nUnable to open file");
                exit(0);
        }
        tot_lines = 0;//count total lines
        ch = fgetc(fp);
        while ( ch != EOF )
        {
                if( ch == 10 ) //new line char
                tot_lines++;
                ch = fgetc(fp);
        }
        fclose(fp);
        //---- take action
        if( strcmp(option,"a")== 0 ) // print all lines
        {
                fp= fopen(fname,"r");
                ch = fgetc(fp);
                while ( ch != EOF )
                {
                        printf("%c",ch);
                        ch = fgetc(fp);
                }
                fclose(fp);
        }
```

```
        else
        {
                printf("\nInvalid option");
                exit(0);
        }
        return(0);
} //typeline
```

2)

```
#include <stdio.h>

#include <stdlib.h>

struct proc_info
{
  int cpub;
  int atime;
  int flag;
} proc[20];

struct rqnode
 { int pid;
   struct rqnode *link;
 } *first=NULL, *curr, *prev, *last;

struct ganttchart
{
  int stime;
  int pid;
  int etime;
```

```
} gchart[20];

int i,n,ctime,gi=0,wtime[20]={0},ttime[20]={0};

void getprocs();
void start();
void addprocq();
void attachtoq(int);
void addgchart(int);
void dispgchart();
void calc_times();
void disptimes();
int finished();
int getfirstproc();
int getetime(int pid);


 main()
{
//clrscr();
  getprocs();
  ctime=0; //set current time to 0
  //randomize();
  start();
  dispgchart();
  disptimes();
}

void getprocs()
{ printf("\nHow many processes: "); scanf("%d",&n);
  printf("\nPID\tCPUB\tA.TIME\n");
```

```
  for (i=1; i<=n; i++)
  { printf("%d\t",i);
    scanf("%d%d",&proc[i].cpub,&proc[i].atime);
    proc[i].flag=0;
   }

}//get_proc

void start()
{ int pid;
  addprocq(); //add procs in rq arrived at ctime
  pid=getfirstproc();//get FCFS process
  while(!finished()) //while not finished all processes
  {
   if(pid!=-1)
   {
    //execute process pid
    addgchart(pid);
    ctime++;  //increment current time
    proc[pid].cpub--;
    if(proc[pid].cpub==0)//proc finished
    {
     //find turn around time
     ttime[pid]+=(ctime-proc[pid].atime);
        pid=getfirstproc();//get FCFS process
    }//if*/
   }//if
   else
   {
    ctime++;//no process in rq
    pid=getfirstproc();//get FCFS process
```

```
   }
   addprocq(); //add procs in rq arrived at ctime
  }//while
  gchart[gi].etime=ctime;
}//start

void addprocq()
{
  for(i=1;i<=n;i++)
  { if(proc[i].atime==ctime)
      attachtoq(i);
   }//for
}//addprocq

void attachtoq(int pid)
{
 curr=(struct rqnode *)malloc(sizeof(struct rqnode));
 curr->pid=pid;curr->link=NULL;
 if(first==NULL)
   first=curr;
 else
   last->link=curr;
 last=curr;
}

int finished() //check all proc finished cpub
{
  for(i=1;i<=n;i++)
  { if(proc[i].cpub!=0)
     return(0);
  }
```

```
  return(1);
}


int getfirstproc()
{
  int pid;
  if(first==NULL)
    return(-1);
  pid=first->pid;
  curr=first; //remove from rq
  first=first->link;
  free(curr);
  return(pid);

}//getfirstproc


void addgchart(int pid)
{static int ppid=-1;
 if(ppid!=pid)
 {
  gchart[++gi].pid=pid;
  gchart[gi].stime=ctime;
  gchart[gi-1].etime=gchart[gi].stime;
  //find waiting time of pid
  wtime[pid]+=(ctime-getetime(pid));
  ppid=pid;
 }
}
```

```
void dispgchart()
{ printf("\n");
  for(i=1;i<=gi;i++)
    printf("|----");
  printf("|\n");

  for(i=1;i<=gi;i++)
    printf("| %d  ",gchart[i].pid);
  printf("|\n");

  for(i=1;i<=gi;i++)
    printf("|----");
  printf("|\n");

  for(i=1;i<=gi;i++)
   printf("%d    ",gchart[i].stime);
  printf("%d\n",gchart[gi].etime);
  //getch();

}

int getetime(int pid)
{
  for(i=gi-1;i>=1;i--)
   if(gchart[i].pid==pid)
      return(gchart[i].etime);
  return(proc[pid].atime);

}

void disptimes()
```

```
{ int sumwt=0,sumtt=0,pid;



  printf("\n**Waiting Time**");
  printf("\nPID\tWtime");
  for(i=1;i<=n;i++)
  {
    printf("\n%d\t%d",i,wtime[i]);
    sumwt+=wtime[i];
  }
  printf("\nAverage:%.2f",(float)sumwt/n);

  printf("\n**Turnaround Time**");
  printf("\nPID\t ttime");
  for(i=1;i<=n;i++)
  {
    printf("\n%d\t%d",i,ttime[i]);
    sumtt+=ttime[i];
  }
  printf("\nAverage:%.2f",(float)sumtt/n);

  //getch();
}
```

Slip21

1)

```
#include<stdio.h>
#include<unistd.h>
#include<stdlib.h>

void main()
```

```
{
   int pid;
   pid=fork();
   if(pid==0)
   {
      printf("I am child process and my id is: %d",getpid());
   }
   else if(pid>0)
   {
      printf("I am parent process and my id is: %d",getppid());
   }
   else
   {
      printf("The process is unsuccessful!");
   }
}
```

2)

```
//simulation of preemptive Priority CPU Scheduling Algorithm

#include <stdio.h>

#include <stdlib.h>

struct proc_info
{
  int atime;
  int cpub;
  int prio;
} proc[20];
```

```
struct rqnode
 { int pid,prio;
   struct rqnode *link;
 } *first=NULL, *curr, *prev, *last;

struct ganttchart
{
  int stime;
  int pid;
  int etime;
} gchart[20];

int i,n,ctime,gi=0,wtime[20],ttime[20];

void getprocs();
void start();
void addprocq();
void attachtoq(int );
void addgchart(int);
void dispgchart();
void disptimes();

void main()
{
  getprocs();
  ctime=0; //set current time to 0
  start();
  dispgchart();
  disptimes();
}
```

```
void getprocs()
{ printf("\nHow many processes: ");
scanf("%d",&n);
  printf("\nPID\tATIME\tCPUB\tPriority\n");
  for (i=1; i<=n; i++)
  { printf("%d\t",i);
    scanf("%d%d%d",&proc[i].atime,&proc[i].cpub,&proc[i].prio);
   }
}//get_proc

void start()
{ int pid;
  addprocq(); //add procs in rq arrived at ctime
  pid=getfirstproc();//get short process
  while(!finished()) //while not finished all processes
  {
    if(pid!=-1)
    { //execute process pid
      ctime++;  //increment current time
      proc[pid].cpub--;
      if(proc[pid].cpub==0)//proc finished
      {
            printf("\nProc %d completed at time %d..",pid,ctime);
            pid=getfirstproc();//get high prio process
      }//if
    }//if
   else
    ctime++;//no process in rq
   addprocq(); //add procs in rq arrived at ctime

   //check if prio of newly arrived proc > prio of current process.
```

```
   //if yes, preempt current process, schedule new process
   if(first!=NULL && first->prio<proc[pid].prio)
   { attachtoq(pid);
     pid=getfirstproc();
   }

  }//while
  gchart[gi].etime=ctime;
}//start

void addprocq()
{
  for(i=1;i<=n;i++)
  { if(proc[i].atime==ctime)
      attachtoq(i);
   }//for
}//addprocq

void attachtoq(int pid)
{
  struct rqnode *selp,*pselp;
  curr=(struct rqnode *)malloc(sizeof(struct rqnode));
  curr->pid=pid;curr->prio=proc[pid].prio; curr->link=NULL;
  if(first==NULL)
  { first=curr;
    return;
  }
  selp=first;
  while(selp->prio<=curr->prio && selp!=NULL)
  { pselp=selp;
    selp=selp->link;
```

```
  }
  if(selp==first)
  { curr->link=first;
    first=curr;
  }
  else
  {pselp->link=curr;
   curr->link=selp;
   }
}

int finished() //check all proc finished cpub
{
  for(i=1;i<=n;i++)
  { if(proc[i].cpub!=0)
     return(0);
  }
  return(1);
}

int getfirstproc()
{
  int pid;
  if(first==NULL)
    return(-1);
  pid=first->pid;
  curr=first; //remove from rq
  first=first->link;
  free(curr);
  addgchart(pid);
  return(pid);
```

```
}//getfirstproc


void addgchart(int pid)
{ static int ppid=-1;
  if(pid!=ppid)
  {
   gchart[++gi].pid=pid;
   gchart[gi].stime=ctime;
   gchart[gi-1].etime=gchart[gi].stime;
   ppid=pid;
  }
}

void dispgchart()
{ printf("\n");

  for(i=1;i<=gi;i++)
    printf("|----");
  printf("|\n");

  for(i=1;i<=gi;i++)
    printf("| %d  ",gchart[i].pid);
  printf("|\n");

  for(i=1;i<=gi;i++)
    printf("|----");
  printf("|\n");

  for(i=1;i<=gi;i++)
```

```
   printf("%d    ",gchart[i].stime);

  printf("%d\n",gchart[gi].etime);



}

void disptimes()
{ int sumwt=0,sumtt=0,pid;
  for(i=1;i<=gi;i++) //calc ttime
  { pid=gchart[i].pid;
    ttime[pid]=(gchart[i].etime-proc[pid].atime);
  }
  for(i=1;i<=gi;i++) //calc wtime
  {
   pid=gchart[i].pid;
   wtime[pid]+=(gchart[i].stime-proc[pid].atime);
   proc[pid].atime=gchart[i].etime;
  }
  printf("\n**Waiting Time**");
  printf("\nPID\tWtime");
  for(i=1;i<=n;i++)
  {
    printf("\n%d\t%d",i,wtime[i]);
    sumwt+=wtime[i];
  }
  printf("\nAverage:%.2f",(float)sumwt/n);

  printf("\n**Turnaround Time**");
  printf("\nPID\t ttime");
  for(i=1;i<=n;i++)
```

```
  {
    printf("\n%d\t%d",i,ttime[i]);
    sumtt+=ttime[i];
  }
  printf("\nAverage:%.2f",(float)sumtt/n);



}
```

Slip22

1)

```
#include<stdio.h>
#include<unistd.h>
#include<stdlib.h>

void main()
{
    int pid,retnice;
    printf("press DEL to stop process\n");
    pid=fork();
    while(1)
    {
        if(pid==0)
        {
            retnice=nice(-5);
            printf("child gets higher CPU priority %d\n",retnice);
            sleep(1);
        }
        else
        {
            retnice=nice(4);
```

```
            printf("Parent gets lower CPU priority %d\n",retnice);
            sleep(1);
        }
    }
}
```

2)

```
//simulation of NP Priority CPU Scheduling Algorithm

#include <stdio.h>

#include <stdlib.h>

struct proc_info
{
  int atime;
  int cpub;
  int prio;
} proc[20];

struct rqnode
 { int pid,prio;
   struct rqnode *link;
 } *first=NULL, *curr, *prev, *last;

struct ganttchart
{
  int stime;
  int pid;
  int etime;
} gchart[20];
```

```
int i,n,ctime,gi=0,wtime[20],ttime[20];

void getprocs();
void start();
void addprocq();
void attachtoq(int );
void addgchart(int);
void dispgchart();
void disptimes();

void main()
{ //clrscr();
  //randomize();
  getprocs();
  ctime=0; //set current time to 0
  start();
  dispgchart();
  disptimes();
}

void getprocs()
{ printf("\nHow many processes: "); scanf("%d",&n);
  printf("\nPID\tATIME\tCPUB\tPriority\n");
  for (i=1; i<=n; i++)
  { printf("%d\t",i);
    scanf("%d%d%d",&proc[i].atime,&proc[i].cpub,&proc[i].prio);
   }
}//get_proc

void start()
```

```
{ int pid;
  addprocq(); //add procs in rq arrived at ctime
  pid=getfirstproc();//get short process
  while(!finished()) //while not finished all processes
  {
    if(pid!=-1)
    { //execute process pid
      ctime++;  //increment current time
      proc[pid].cpub--;
      if(proc[pid].cpub==0)//proc finished
      {
            printf("\nProc %d completed at time %d..",pid,ctime);
            ttime[pid]=ctime-proc[pid].atime;
            pid=getfirstproc();//get high prio process
      }//if
    }//if
   else
    ctime++;//no process in rq

   addprocq(); //add procs in rq arrived at ctime
  }//while
  gchart[gi].etime=ctime;
}//start

void addprocq()
{
  for(i=1;i<=n;i++)
  { if(proc[i].atime==ctime)
      attachtoq(i);
   }//for
}//addprocq
```

```
void attachtoq(int pid)
{
  struct rqnode *selp,*pselp;
  curr=(struct rqnode *)malloc(sizeof(struct rqnode));
  curr->pid=pid;curr->prio=proc[pid].prio; curr->link=NULL;
  if(first==NULL)
  { first=curr;
    return;
  }
  selp=first;
  while(selp->prio<=curr->prio && selp!=NULL)
  { pselp=selp;
    selp=selp->link;
  }
  if(selp==first)
  { curr->link=first;
    first=curr;
  }
  else
  {pselp->link=curr;
   curr->link=selp;
   }
}

int finished() //check all proc finished cpub
{
  for(i=1;i<=n;i++)
  { if(proc[i].cpub!=0)
     return(0);
  }
```

```
  return(1);
}

int getfirstproc()
{
  int pid;
  if(first==NULL)
    return(-1);
  pid=first->pid;
  curr=first; //remove from rq
  first=first->link;
  free(curr);
  addgchart(pid);
  wtime[pid]=ctime-proc[pid].atime;
  return(pid);

}//getfirstproc



void addgchart(int pid)
{
   gchart[++gi].pid=pid;
   gchart[gi].stime=ctime;
   gchart[gi-1].etime=gchart[gi].stime;
}

void dispgchart()
{ printf("\n");

  for(i=1;i<=gi;i++)
    printf("|----");
```

```
  printf("|\n");

  for(i=1;i<=gi;i++)
    printf("| %d  ",gchart[i].pid);
  printf("|\n");

  for(i=1;i<=gi;i++)
    printf("|----");
  printf("|\n");

  for(i=1;i<=gi;i++)
   printf("%d    ",gchart[i].stime);

  printf("%d\n",gchart[gi].etime);
 // getch();

}

void disptimes()
{ int sumwt=0,sumtt=0,pid;

  printf("\n**Waiting Time**");
  printf("\nPID\tWtime");
  for(i=1;i<=n;i++)
  {
    printf("\n%d\t%d",i,wtime[i]);
    sumwt+=wtime[i];
  }
  printf("\nAverage:%.2f",(float)sumwt/n);

  printf("\n**Turnaround Time**");
```

```
  printf("\nPID\t ttime");
  for(i=1;i<=n;i++)
  {
    printf("\n%d\t%d",i,ttime[i]);
    sumtt+=ttime[i];
  }
  printf("\nAverage:%.2f",(float)sumtt/n);

  //getch();
}
```

Slip23

1)

```
#include<stdio.h>
#include<unistd.h>
#include<stdlib.h>

void main()
{
    int pid;
    pid=fork();
    if(pid>0)
    {
        printf("I am parent process and my id is:",getppid());
    }
    else if(pid==0)
    {
        printf("I am child process and my id is:",getpid());
        printf("My parent id before termination is:",getppid());
        sleep(5);
        printf("My parent id after termination is:",getppid());
```

```
    }
    else
    {
        printf("The process is unsuccessful!");
    }
}
```

2)

Slip24

1)

```
#include<stdio.h>
#include<sys/types.h>
#include<unistd.h>
#include<stdlib.h>

void bubblesort(int arr[30], int n)
{
    int i, j, temp;
    for(i=0; i<n; i++)
    {
        for(j=0; j<n-1; j++)
        {
            if(arr[j]>arr[j+1])
            {
                temp=arr[j];
                arr[j]=arr[j+1];
                arr[j+1]=temp;
            }
        }
    }
}
```

```
void insertionsort(int arr[30], int n)
{
   int i, j, temp;
   for(i=1; i<n; i++)
   {
      temp=arr[i];
      j=i-1;
      while(j>=0 && temp<=arr[j])
      {
         arr[j+1]==arr[j];
         j=j-1;
      }
      arr[j+1]=temp;
   }
}

void main()
{
   int arr[25], arr[25], n, i, status;
   printf("\nEnter the number of values in array:");
   scanf("%d",&n);
   printf("\nEnter the array elements:");
   for(i=0; i<n; i++)
      scanf("%d",&arr[i]);
   int pid=fork();
   if(pid==0)
   {
      sleep(2);
      printf("\nchild process\n");
      printf("child process id=%d\n",getpid());
```

```
        insertionsort(arr, n);
        printf("\nElements sorted using insertionsort:");
        printf("\n");
        for(i=0; i<n; i++)
            printf("%d",&arr[i]);
        printf("\b");
        printf("\nparent process id=%d\n",getppid());
    }
    else
    {
        printf("\nparent process\n");
        printf("parent process id=%d\n",getpid());
        bubblesort(arr, n);
        printf("Elements sorted using bubblesort:");
        printf("\n");
        for(j=0; j<n; j++)
            printf("%d",&arr[j]);
        printf("\n\n\n");
    }
}
```

2)

```
#include <stdio.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <dirent.h>
//#include <bfd.h>
char comm[100],*ptr;
```

```
char *args[10];
int tot_arg;

int count( char option, char *fname);

main()
{
	do
	{
		printf("myshell $ ");
		getcomm();
		sep_arg();
		take_action();
	}while(1);
} // main

sep_arg()
{
	int i,j;
	i = j = tot_arg = 0;

	args[tot_arg] = (char*)malloc(sizeof(char) * 20);
	for(i=0; comm[i] !='\0' ; i++)
	{
		if( comm[i] == ' ')
		{
			args[tot_arg][j] = '\0';
			tot_arg++;
			args[tot_arg] = (char*)malloc(sizeof(char) * 20);
			j=0;
		}
```

```
                else
                {
                        args[tot_arg][j] =comm[i];
                        j++;
                }
        } // while
        args[tot_arg][j] = '\0'; // complete last word
        return;
}

getcomm()
{
        int len;
        ptr = fgets(comm,80,stdin);
        len = strlen(comm);
        comm[len-1] = '\0';
        return;
} // getcomm

take_action()
{
        char str[100];
        pid_t   pid;
        int status;

        if(strcmp(args[0],"count") == 0 )
                count(*args[1],args[2]);
        else
        {
                if( ptr == NULL )
                        exit(0);
```

```
                if ( strlen(comm) <= 1 )
                        return;
                if( ( pid = fork() ) < 0 )
                {
                        printf("\nUnable to create process");
                }
                else
                if(  pid == 0 ) // child
                {   sscanf(comm,"%s",comm);
                        execvp(comm,args);
                        strcpy(str,"./");
                        strcat(str,args[0]);
                        execvp(str,args);
                        printf("\n%s : command not found",comm);
                        exit(0);
                }
                else //parent
                 waitpid(pid,&status,0);
        } // else
}// take action

int count( char option, char *fname)// count c/w/l   <filename>
{
        int ccnt,wcnt,lcnt,wflag;
        int ch;
        FILE *fp;
        int i;
        fp= fopen(fname,"r");
        if( fp == NULL )
        {
                printf("\nUnable to open file");
```

```
        exit(0);
}
ccnt =wcnt = lcnt = 0;
wflag=1;
ch = fgetc(fp);
while ( ch != EOF)
{
        ccnt++;
        if( ch ==' ' || ch== '\t')
        {
                if( wflag == 0 )
                {
                        wcnt++;
                        wflag = 1;
                }
        }
        else
        if( ch == '\n')
        {
                lcnt++;
                if( wflag == 0 )
                {
                        wcnt++;
                        wflag = 1;
                }
        }
        else
        {
                wflag = 0;
        }
        ch = fgetc(fp);
```

```
        }//while
        if( wflag == 0 )
        {
                wcnt++;
                lcnt++;
        }
        fclose(fp);
        switch(option)
        {
        case 'c' : // print all chars
                        printf("\nTot chars = %d\n",ccnt);
                        break;
        case 'w' : // print all words
                        printf("\nTot words = %d\n",wcnt);
                        break;
        case 'l' : // print all lines
                        printf("\nTot lines = %d\n",lcnt);
                        break;
        } // switch
        return;
}// count
```

Slip25

1)

2)

```
#include <stdio.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/stat.h>
```

```
#include <fcntl.h>
#include <dirent.h>
//#include <bfd.h>
char comm[100],*ptr;
char *args[10];
int tot_arg;

int search(char option, char *pattern, char *fname);

main()
{
	do
	{
		printf("myshell $ ");
		getcomm();
		sep_arg();
		take_action();
	}while(1);
} // main

sep_arg()
{
	int i,j;
	i = j = tot_arg = 0;

	args[tot_arg] = (char*)malloc(sizeof(char) * 20);
	for(i=0; comm[i] !='\0' ; i++)
	{
		if( comm[i] == ' ')
		{
			args[tot_arg][j] = '\0';
```

```
                        tot_arg++;
                        args[tot_arg] = (char*)malloc(sizeof(char) * 20);
                        j=0;
                }
                else
                {
                        args[tot_arg][j] =comm[i];
                        j++;
                }
        } // while
        args[tot_arg][j] = '\0'; // complete last word
        return;
}

getcomm()
{
        int len;
        ptr = fgets(comm,80,stdin);
        len = strlen(comm);
        comm[len-1] = '\0';
        return;
} // getcomm

take_action()
{
        char str[100];
        pid_t   pid;
        int status;

        if(strcmp(args[0],"search") == 0 )
                search(*args[1],args[2],args[3]);
```

```
        else
        {
                if( ptr == NULL )
                        exit(0);
                if ( strlen(comm) <= 1 )
                        return;
                if( ( pid = fork() ) < 0 )
                {
                        printf("\nUnable to create process");
                }
                else
                if(  pid == 0 ) // child
                {   sscanf(comm,"%s",comm);
                        execvp(comm,args);
                        strcpy(str,"./");
                        strcat(str,args[0]);
                        execvp(str,args);
                        printf("\n%s : command not found",comm);
                        exit(0);
                }
                else //parent
                 waitpid(pid,&status,0);
        } // else
}// take action

int search(char option, char *pattern, char *fname)
{
        char buff[100],*ptr;
        int cnt;
        FILE *fp;
        int i;
```

```
        fp= fopen(fname,"r");
        if( fp == NULL )
        {
                printf("\nUnable to open file");
                exit(0);
        }
        switch(option)
        {
           case 'f' : //print first occ
                        while ( !feof (fp))
                        {
                                buff[0] = '\0';
                                fgets(buff,80,fp);
                                ptr = strstr(buff,pattern);
                                if( ptr != NULL )
                                {
                                        printf("%s",buff);
                                        break;
                                }
                        }
                        fclose(fp);
                        break;
        } // switch
        return;
}//search
```